

DEDICATION

TO THE MBISSICK FAMILY

ACKNOWLEDGEMENT

Drafting this document would not have been possible without the invaluable contributions of several individuals who dedicated their time and effort to ensure its completion. I extend my heartfelt gratitude to the following people:

- ❖ **The Almighty God, JEHOVAH**, for granting me the strength and blessings to reach this stage of my life, and for guiding me through challenging moments when I felt like giving up.
- ❖ **Mr. Armand Claude ABANDA**, the Resident Representative of IAI-Cameroon, for his unwavering support, words of encouragement and insightful advice on navigating various situations.
- ❖ **Mrs. TCHOUTOUO Isabelle**, our Academic supervisor, for her constant moral support, advice, and guidance at every step of my academic year.
- ❖ **Mr. AGBOR Anderson, Mr. MESSIO Ulrich, Mr. NGUH Prince, Mr. ASSANE Derick**, my academic teachers, for their valuable advice and assistance in the preparation of this document.
- ❖ To all my teachers at AICS, for imparting the knowledge that has brought me to this point; I am deeply thankful to each of you.
- ❖ **To my mother, Mm. MBISSICK Esther Raissa**, your love and unwavering support have been my greatest strength. Thank you for believing in me and for your endless encouragement.
- ❖ **To Mr. MBOULE DJIKE GUY Patrick C.**, who has been like a father to me, I am immensely grateful for your guidance and the significant role you have played in my education.
- ❖ To my grandmother, **Mm. OLLO Claire Veuve MBISSICK**, your unwavering care and wisdom have been a source of inspiration. Thank you for your nurturing presence in my life.
- ❖ To all my aunts, for their countless assistance and the caring support they provided throughout my journey.
- ❖ To my brothers and sisters, for their encouragement, love, and unwavering support.
- ❖ To all my friends and classmates at AICS, for their contributions and the tremendous help they provided in acquiring essential skills.
- ❖ Each of you has played a significant role in my journey, and I am profoundly grateful for your support. Thank you all!

Contents

DEDICATION	1
ACKNOWLEDGEMENT.....	2
ABREVIATIONS	4
LIST OF FIGURES	5
LIST OF TABLES	7
ABSTRACT	8
Résumé	9
GENERAL INTRODUCTION	10
BOOK I : EXISTING SYSTEM	27
BOOK II : SPECIFICATION BOOK	37
BOOK III : ANALYSIS BOOK	54
BOOK IV : CONCEPTION BOOK.....	88
BOOK V: REALIZATION PHASE	107
BOOK VI: TEST OF FUNCTIONALITIES	118
BOOK VII: USER GUIDE	124
GENERAL CONCLUSION.....	137
ANNEXES	xiii
BIBLIOGRAPHY	xiv
WEBOGRAPHY	xv
TABLE OF CONTENTS	xvi

ABBREVIATIONS

- ❖ **2TUP:** Two Track Unified Process
- ❖ **AICS:** African Institute of Computer Sciences
- ❖ **API:** Application Programming Interface
- ❖ **CSS:** Cascading Style Sheet
- ❖ **DBMS:** Database Management System
- ❖ **HTTP:** Hypertext Transfer Protocol
- ❖ **IS:** Information System
- ❖ **JWT:** Java Web Token
- ❖ **MVC:** Model View Controller
- ❖ **UML:** Unified Modelling Language

LIST OF FIGURES

<i>Figure 1: Gantt Chart</i>	<i>50</i>
<i>Figure 2: UML diagram overview</i>	<i>59</i>
<i>Figure 3: 2TUP diagram.....</i>	<i>62</i>
<i>Figure 4: Use Case Diagram of formalism.....</i>	<i>65</i>
<i>Figure 5: General use case diagram.....</i>	<i>72</i>
<i>Figure 6: specific use case of manage products</i>	<i>73</i>
<i>Figure 7: make Payment use case diagram</i>	<i>73</i>
<i>Figure 8: Communication Diagram Formalism</i>	<i>74</i>
<i>Figure9: Communication Diagram of authentication</i>	<i>76</i>
<i>Figure 10: Vote communication diagram</i>	<i>77</i>
<i>Figure 11: sequence Diagram Formalism</i>	<i>78</i>
<i>Figure 12: Sequence Diagram of authentication</i>	<i>80</i>
<i>Figure 13: Sequence diagram to vote price</i>	<i>81</i>
<i>Figure 14: Activity Diagram formalism.....</i>	<i>82</i>
<i>Figure 15: Activity diagram of authentication.....</i>	<i>85</i>
<i>Figure 16: Hardware Diagram.....</i>	<i>91</i>
<i>Figure 17: Physical Architecture</i>	<i>92</i>
<i>Figure 18: The MVC architecture.....</i>	<i>93</i>
<i>Figure 19: Class Diagram formalism</i>	<i>94</i>
<i>Figure 20: Class diagram</i>	<i>97</i>
<i>Figure 21: Formalism of a state machine diagram</i>	<i>99</i>
<i>Figure 22: State machine Diagram of.....</i>	<i>101</i>
<i>Figure 23: State machine diagram of rent property</i>	<i>102</i>
<i>Figure 24: Package Diagram Formalism</i>	<i>103</i>
<i>Figure 25: TchopPrice Package Diagram</i>	<i>105</i>
<i>Figure 26: Deployment diagram Formalism</i>	<i>110</i>
<i>Figure 27: TchopPrice Deployment Diagram</i>	<i>112</i>
<i>Figure 28: Formalism of a com2ponent Diagram</i>	<i>113</i>
<i>Figure 29: login test.....</i>	<i>123</i>
<i>Figure 30: create property test</i>	<i>123</i>
<i>Figure 31: MongoDB Logo.....</i>	<i>127</i>
<i>Figure 32 Downloading MongoDB community version</i>	<i>127</i>

<i>Figure 33 Launching the installation wizard</i>	128
<i>Figure 34 Clicking on next</i>	128
<i>Figure 35: Agreement license</i>	129
<i>Figure 36: Select complete setup</i>	129
<i>Figure 37: Run Service network user</i>	130
<i>Figure 38: Selecting MongoDB compass and clicking next</i>	130
<i>Figure 39: Click on install to start installation.....</i>	131
<i>Figure 40: Click on finish to complete installation.....</i>	131
<i>Figure 41: Landing Page</i>	132
<i>Figure 42: login page.....</i>	133
<i>Figure 43: Admin dashboard</i>	134

LIST OF TABLES

<i>Table 1: limitations of the existing system</i>	Error! Bookmark not defined.
<i>Table 2: Hardware Resources.....</i>	45
<i>Table 3: Software Resources of the project</i>	46
<i>Table 4: Human Resources</i>	47
<i>Table 5: Total Project Estimation Cost.....</i>	48
<i>Table 6: chronogram of activities</i>	49
<i>Table 7 : list of participants</i>	52
<i>Table 8: Difference between UML and MERISE</i>	60
<i>Table 9: use case Diagram Component</i>	66
<i>Table 10: Actors of our system.....</i>	68
<i>Table 11: secondary actors of our system.....</i>	68
<i>Table 12: Textual Description of Authenticate</i>	69
<i>Table 13 : Textual Description of make payment</i>	70
<i>Table 14: Communication Diagram Component</i>	75
<i>Table 15: Sequence Diagram Components</i>	79
<i>Table 16: Components of an activity diagram</i>	83
<i>Table 17: Components of a class diagram.....</i>	95
<i>Table 18: Relationship between classes</i>	98
<i>Table 19: State machine Diagram Components</i>	100
<i>Table 20: Package Diagram Components</i>	104
<i>Table 21: Components of a Deployment Diagram.....</i>	111
<i>Table 22: Elements of a component diagram.....</i>	114
<i>Table 23: Software tools used for the project</i>	115
<i>Table 24: Presentation of the languages used for the software</i>	116

ABSTRACT

In today's evolving real estate landscape, access to inclusive and decentralised investment opportunities remains limited for the average citizen. This project focuses on the design and implementation of a blockchain-powered web application that enables the tokenisation of real estate assets and the automated distribution of rental income through smart contracts. Centred around the Cameroonian context, the platform aims to democratise property investment by allowing individuals to purchase digital shares of buildings and earn passive income, starting with minimal capital. Leveraging modern technologies such as NestJS, React.js, MongoDB, and Solidity, the system is structured to handle property listing, token sale, rental collection, and investor payout seamlessly. Property owners benefit from an alternative fundraising mechanism without relying on traditional banking, while investors gain secure, transparent access to real estate returns. Tenants, in turn, can pay rent via integrated local payment APIs, ensuring digital traceability and ease of use. A significant phase of the project will involve analytical modelling using UML to define system architecture, user roles, and interaction workflows. The development process will follow Agile principles, particularly the Scrum methodology, to promote iterative design, continuous integration, and adaptability to user feedback. Ultimately, this platform aspires to serve as a real-world case study in the practical application of blockchain technology to real estate, showcasing how digital tools can address financial exclusion and modernise property management in African markets.

Keywords

- Agile Development
- Blockchain
- MongoDB
- NestJS
- Property Tokenisation
- React.js
- Real Estate
- Smart Contracts
- Solidity
- UML
- Web Application

Résumé

Dans le secteur Immobilier actuel, de plus en plus dynamique, l'accès à des informations transparentes et en temps réel sur les est essentiel pour les Investisseurs et Propriétaire et les parties prenantes. Ce projet porte sur la conception et la mise en œuvre d'une application web centrée sur l'investissement dans l'immobilier, visant à surveiller les prix des produits agricoles dans différentes régions. Grâce à l'utilisation de **Node.js/Nest.js**, **React.js** et **MongoDB**, la plateforme permettra aux agriculteurs de consulter des données actualisées sur les prix, leur offrant ainsi les moyens de prendre des décisions éclairées et de renforcer leur pouvoir de négociation.

L'application répond aux défis majeurs rencontrés par de nombreux agriculteurs au Cameroun, tels que la volatilité des prix, le manque de transparence du marché et un accès limité aux outils numériques. Grâce à une interface utilisateur intuitive, les utilisateurs pourront consulter les tendances historiques, comparer les prix entre différents marchés et recevoir des alertes en cas de fortes fluctuations.

Une phase clé du projet est l'analyse technique, durant laquelle les modèles **UML** seront utilisés pour modéliser l'architecture du système, les flux de travail et les interactions utilisateur. Ces modèles guideront le développement afin de garantir une réponse adéquate aux besoins réels du terrain.

Pour le backend, **Node.js** offrira un environnement robuste et évolutif pour construire une API performante, tandis que **MongoDB** assurera la gestion flexible des données agricoles semi-structurées. **React.js** alimentera l'interface utilisateur, assurant une interaction rapide et fluide. Le développement sera conduit selon les méthodes **UML**, en particulier avec le cadre **2TUP**, afin de permettre des ajustements rapides et continus tout au long du cycle de vie du projet.

L'objectif ultime de cette plateforme est de renforcer les capacités des agriculteurs locaux, en augmentant la transparence du marché, en favorisant des décisions basées sur les données et en comblant le fossé informationnel qui freine encore le développement agricole en zones rurales.

Mots-clés

- Agile Development
- Blockchain
- Immobilier
- MongoDB
- NestJS
- Property Tokenisation
- React.js
- Smart Contracts
- Solidity
- UML
- Web Application

GENERAL INTRODUCTION

In a rapidly digitalising world, Cameroon's real estate sector remains largely traditional, with limited access to property ownership and inefficient rent management. As part of my Level III end-of-year project at AICS Cameroon, I have chosen the theme: **“Design and Implementation of a Tokenised Real Estate Platform.”** This project aims to create a web-based system that allows property owners to convert buildings into digital tokens and sell them to investors, who will then earn monthly rent income distributed automatically through blockchain smart contracts. Developed using React.js, NestJS, MongoDB, and Solidity, the platform seeks to make real estate investment more accessible, transparent, and inclusive by combining modern technologies with local economic realities. The report is structured into **eight (8)** key sections as outlined below:

1. **Insertion Document** – In this book, we shall present the company in which we spent our internship period and the way we were welcomed in the company.
2. **Existing System** – This section presents the current means by which farmers obtain price information and outlines the problems and limitations involved.
3. **Specification Book** – Defines system requirements, user expectations, and constraints related to time, cost, and resources.
4. **Analysis Document** – Describes the chosen analysis methods and includes all UML diagrams used to understand and model the system.
5. **Conception Phase** – Details both general and specific design choices made to build the system architecture.
6. **Realisation Phase** – Explains the development and implementation steps using the selected technologies.
7. **Test of Functionalities** – Demonstrates how various modules of the application function and interact.
8. **User Guide** – Provides comprehensive instructions on system usage, including setup and key user actions.

BOOK I : INSERTION PHASE

SUMMARY

The insertion phase refers to the initial stage of an internship where the intern is introduced to the organization, its structure, and their role. The insertion phase is crucial for helping the intern feel welcomed, prepared, and confident in their ability to contribute effectively to the organization during the internship. It lays the foundation for a successful and productive experience. The professional integration phase is the next stage in the internship process, where the intern actively participates and contributes to the organization's operations. This phase focuses on the intern's ability to apply their knowledge and skills in a real-world professional setting.

Preamble

This phase presents the details of how we were integrated in the host company, the company presentation and organisation.

Content

INTRODUCTION

- I. WELCOME AND INTERGRATION
- II. GENERAL PRESENTATION OF MICKMAQ
- III. ORGANISATION OF MICKMAQ
- IV. ADMINISTRATION AND FUNCTIONAL ORGANISATIONAL OF MICKMAQ
- V. HARDWARE RESOURCES OF MICKMAQ
- VI. SOFTWARE RESOURCES OF MICKMAQ
- VII. BRIEF PRESENTATION OF THE PROJECT THEME

CONCLUSION

INTRODUCTION

The professional integration phase follows the insertion phase in the internship process, during which the intern actively participates in and contributes to the organization's operations. This stage centres on the intern's ability to apply their knowledge and skills within a real-world professional context. Integration into the organizational structure is a crucial step for the development and well-being of both the intern and the employee; this period typically lasts two weeks. In this section, we will first discuss the reception process and then provide an overview of the organizational structure in which we have integrated.

I. WELCOME AND INTERGRATION

Welcome

We arrived at Mickmaq on Monday, 14th of July 2025, at 8:00 a.m. We were welcomed by Mr. **MEFIRE FOUAPON HASSAN**, the enterprise software development engineer and Director, who introduced us to our workspace and his collaborators. Later, we were assigned internship themes to work on.

Integration

A working day at Mickmaq starts from 8:00 a.m. to 5:00 p.m. Our tasks in the company generally follow a predefined routine. Every day, we practiced different languages and tools needed for our themes/projects. It was also an opportunity to present our difficulties and challenges to everyone and receive directives. During the first week, we verified concepts with basic java, OOP concepts, HTML and CSS to ensure everyone was at the same level before kicking off. We discussed project ideas and were advised to propose our own. we proceeded with verification of JavaScript concepts through some exercises to test our reasoning and understanding faculties. We also discussed the specification book with our professional supervisor and its different components, with him explaining what was required from us in each component. We further discussed our different project to give them shape.

II. GENERAL PRESENTATION OF THE MICKMAQ

History

Mickmaq LLC, headquartered in Yaoundé Cameroon, is an E-commerce and online retailing company predominantly founded by **Mounchingam Zounkaraneni** in 2019 with its General Director **Fouapon Hassan**.

Aims

Mickmaq aims to be the first E-commerce platform whose primary focus is to offer a reputable digital marketplace for wholesalers. Both consumers and merchants can connect to buy and sell from each other, and drivers connect to ensure delivery of wholesale products.

Goal

Mickmaq's goal is to provide a platform for wholesalers, distributors, drivers and small businesses that are ready to leverage Omni-channel commerce strategies to grow their sales. We strive to improve the overall online wholesale shopping and delivery experience, hence, stimulating the growth of small businesses in Africa.

Activities

- It operates a digital marketplace where wholesalers and distributors can sell wholesale products in a wide range of categories, including groceries, tools, construction products, appliances, and services.
- Software development and maintenance and Training in Software related fields

Geographical location

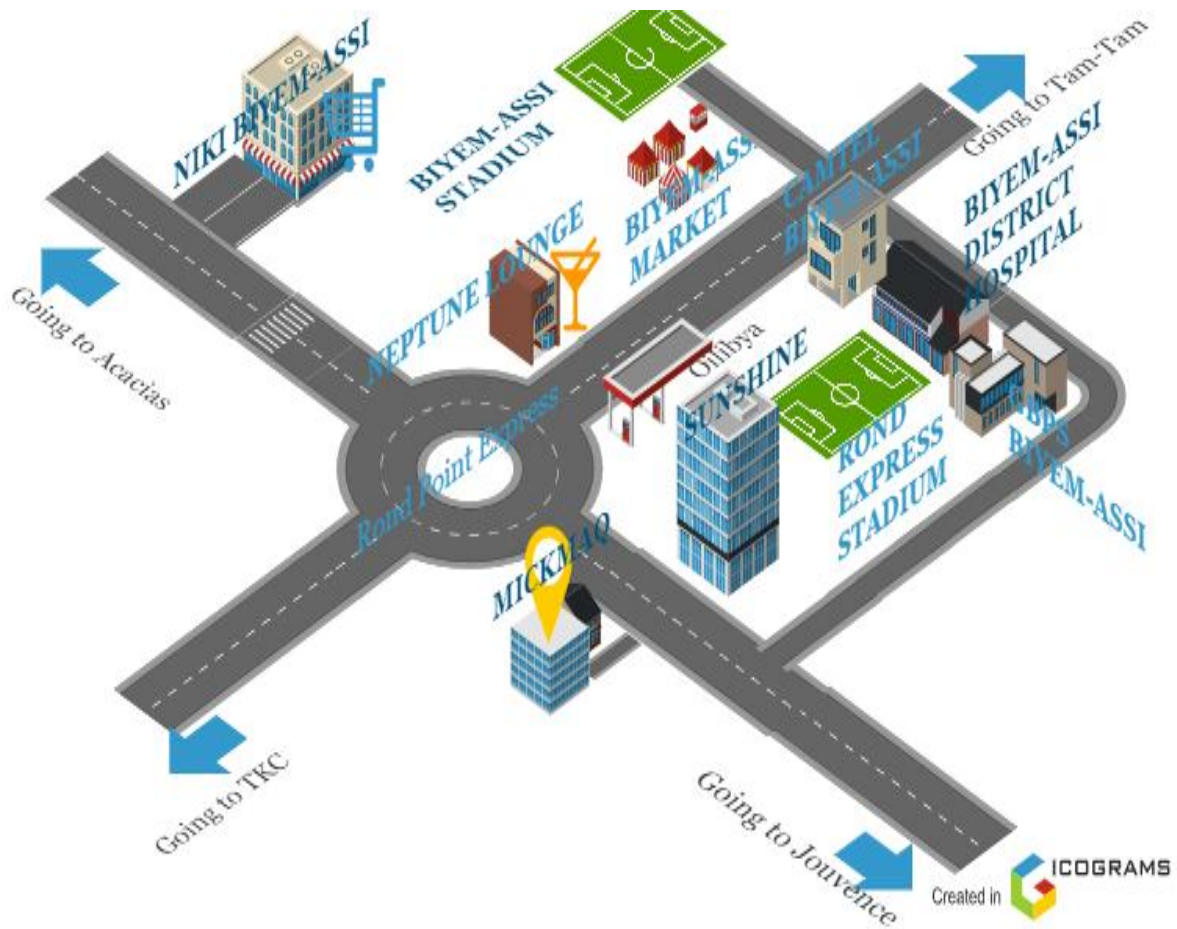


Figure 1 : geographical location of MICKMAQ

Company Identification form

Table 1: company identification form

IDENTIFICATION	DESCRIPTION
Headquarters	Yaoundé « Rond-point express »
Main Activity	E-Commerce
Creation Date	2019, Cameroon
Services	- Web Development - Mobile Development - Digital Marketing - Customer Relationship Management
Type of Company	Private
Contact	Telephone : 6 80 051 785
Email	info@mickmaq.com
Web Site	https://www.mickmaq.com
Legal Form	SARL
Chief Executive Office	M. Fomenky Ache
Language Spoken	French and English

III. Administrative and Functional Organisation of Mickmaq

Mickmaq is headed by the top executive in the company and is responsible for providing overall leadership, vision, and strategic direction. Their role involved setting company objectives and making key decisions. Mickmaq is organized as follows.

A. The Country Director

Each country has its director and for Cameroon Mickmaq has a director which has the following mission.

- He oversees the operations of Mickmaq within the country.
- Managing local teams, coordinating with headquarters, and ensuring the company's objectives are met in the designated market.
- They establish and maintain relationships with clients, partners, and government entities in the country, and may be involved in business development, sales, and marketing activities.

B. The Technical Director

He is responsible for the technical aspects of Mickmaq's operations, his mission in the company is the following:

- He oversees the technical team, including software developers and engineers, and provides guidance to technology choices, software architecture and development methodologies.
- He ensures the delivery of high-quality software solutions, foster innovation and keeps up with industry trends.

C. Administrative and financial Services

He oversees handling all administrative and financial function within the company and their mission includes:

- Managing human resources such as recruitment, employee relations and performances management
- He also handles financial management, including budgeting, financial reporting, accounts payable and receivable and the overall financial planning and analysis of the company.

D. Study and Development Services

These are mainly the developers, they are responsible for conducting research, analysis, and feasibility studies for software development projects.

- They work closely with clients to understand their requirements, propose solutions and develop project plans.
- This team also carries out the actual development of software solutions, including coding, testing, and implementation.

Mickmaq's Organigram

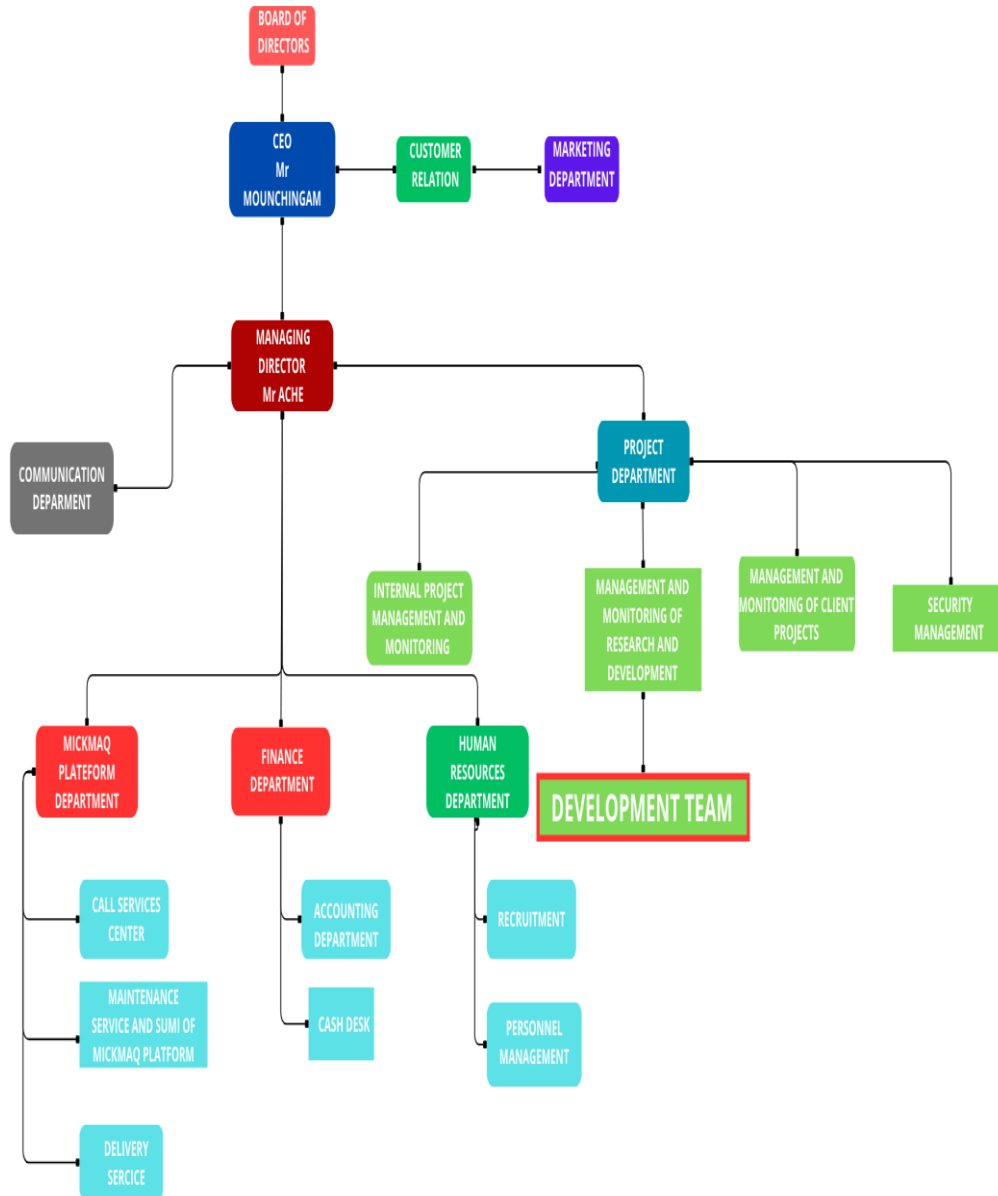


Figure 2 organisational chart of MICKMAQ

I. Hardware Resources

Table 2: Hardware resources of Mickmaq

No	NAME	BRAND	QUANTITY
1	Optical fibre	MTN	01
2	LCD Screen	DELL	02
3	Training equipment and office furniture	Anderson's Office Furniture (AoF)	100

I. Software Resources

Table 3: Software resources of Mickmaq

SOFTWARE RESOURCES	
Operating system	Windows 10,11 and ubuntu
Integrated Development Environment	Eclipse, visual studio, idea IntelliJ
Project management tools	Trello , Sentry and Gmail for the maintenance and follow up
Version control	Git for the version control and git fabricator
Database Management System	PostgreSQL, SQLite, MongoDB
Testing and Quality assurance	Postman and swagger
Deployment	Google cloud platform
Programming languages	React JS, Angular JS, React Native, Flutter for the frontend and Spring boot, Node JS, Nest JS, for the backend

I. BRIEF PRESENTATION OF THE PROJECT THEME

“Design and Implementation of a Tokenised Real Estate Platform” The aim of this project is to design a digital platform that introduces a new method of investing in real estate through **tokenisation**, supported by blockchain technology. This concept enables property owners to divide their assets into digital shares (tokens) and sell them to multiple investors. Investors, in turn, receive monthly rent income proportional to their holdings. This system promotes economic inclusion, transparency, and accessibility within the real estate sector, particularly for individuals with limited capital. Extensive preliminary research was carried out to understand the economic, social, and technical dimensions of this project. The platform will be developed using **React.js**, **NestJS**, **MongoDB**, and **Solidity**, supported by local payment APIs for rent transactions and smart contracts for automated rent distribution.

CONCLUSION

In conclusion the insertion phase is that part where the student is introduced to professional life. It is a very educational part, and this made me to know about the school how it came to exist and how it expanded over the years. we got to know the department and its staff how every department functions. This made us feel very comfortable and well welcomed. After all these we tried to select a theme which we will work on during the internship after discussion we decided to “**DESIGN AND IMPLEMENTATION OF A TOKENISED REAL ESTATE PLATFORM**” Now that the theme has been chosen, we can now work on the specification book of our report where the objectives and requirements will be identified.

BOOK II : EXISTING SYSTEM

Preamble

This document outlines the comprehensive specifications for our project, detailing functional and non-functional requirements, UI design, system limitation, and database structures serving as the development guide.

Content

INTRODUCTION

- I. PRESENTATION OF THE THEME
- II. STUDY OF THE EXISTING SYSTEM
- III. CRITICISM OF THE EXISTING SYSTEM
- IV. PROBLEMATICS
- V. PROPOSED SOLUTION

CONCLUSION

INTRODUCTION

As software engineers, our role is to create innovative solutions that address real-world challenges through technology. Before proposing solutions, we must first understand the limitations of current systems: Why do these problems exist? What methods are currently used to manage them? And where do the gaps lie? In the case of real estate investment and rental income distribution in Cameroon, the current system is inaccessible, highly manual, and excludes most of the population. Exploring this landscape allows us to build technology that is both socially relevant and technically feasible

I. PRESENTATION OF THE PROJECT THEME

The theme of this project is “**Design and Implementation of a Tokenised Real Estate Platform.**” It addresses the lack of inclusive and transparent access to property investment in Cameroon, where real estate remains reserved for the wealthy few. The proposed solution, **Green**, is a web-based platform that enables property owners to digitise and sell fractional ownership (tokens) of their real estate assets. These tokens allow individuals to invest with as little as 50,000 FCFA and earn passive income through monthly rent, distributed automatically via smart contracts. Developed using React.js, NestJS, MongoDB, and Solidity, Green aims to democratise real estate investment, modernise rent distribution, and support the digital transformation of Cameroon’s financial and property sectors.

II. DESCRIPTION OF THE EXISTING SYSTEM

The real estate sector in Cameroon remains predominantly informal, with property ownership, investment, and rental income opportunities largely accessible to a privileged few. High capital requirements, complex legal documentation, and limited access to formal land titles exclude most ordinary citizens—especially young professionals, students, and low-income earners—from participating in wealth-building through real estate.

Urbanization is driving strong demand: Over **56.5% of Cameroon's population lived in urban areas by 2023**, with urban centres like Douala and Yaoundé experiencing rapid growth. This growth fuels demand for housing, yet the country faces a significant housing deficit estimated at **about 2.5 million units**, creating intense pressure on the housing market. The government's social housing programs, initiated since 2009, have had limited impact on closing this gap, with slow construction and persistent affordability challenges

1. Existing Sources of Price Information

Farmers currently rely on several informal methods to obtain market prices:

- **Full Ownership Model:** Typically, individuals, especially property buyers, must purchase entire buildings or land plots outright. Given the high prices—some housing units priced between 30 million to 836 million FCFA—and lack of formal credit access, this is financially prohibitive for most.
- **Informal Co-ownership:** Some groups pool resources to invest jointly and share profits manually, but these arrangements lack formal contracts, transparency, and automated processes, leading to disputes and inefficiencies.
- **Manual Rent Management:** Rent collection is mostly conducted directly by owners in cash or via mobile money, without centralized tracking or automation, creating risks of errors, delays, and mistrust.
- **Real Estate Agencies:** While agencies exist, high service fees and limited trust reduce their adoption among average citizens.

Average monthly rent for a one-bedroom apartment in city centres like Yaoundé or Douala is approximately 456,188 FCFA, and around 287,722 FCFA outside the centre, reflecting both high demand and limited affordable supply

2. Common Challenges in the Current System

The current system presents multiple challenges:

- **High Cost of Entry:** Investment requires large capital and formal land titles, which fewer than 15% of land properties possess, limiting accessibility.
- **No Access to Passive Income:** Young workers and savers have no reliable means to earn rental income unless they own full properties.
- **Lack of Transparency:** Informal rent sharing and co-investment arrangements are often unstructured and non-transparent, causing mistrust.
- **Manual Processes:** Rent collection, distribution, and record-keeping are mostly manual, increasing risk of errors and disputes.
- **Limited Market Visibility:** Most properties are not listed on centralized platforms, and there are no digital tools for fractional ownership or automated rent management.
- **Housing Deficit and Affordability:** The estimated housing shortage of 2.5 million units, combined with rising land prices especially in urban hubs, limits adequate supply of affordable housing

3. A Typical Investor's Experience

Take for example a young professional in Yaounde who wishes to invest in real estate. With a monthly income of 150,000 FCFA, he cannot afford to buy property outright. His only options are to save for years or join an unreliable tontine. Even if he manages to invest with others in a shared building, rent distribution is slow, undocumented, and often dependent on informal arrangements. There is no platform that allows him to invest small amounts in verified properties, receive regular rent, and monitor everything online.

This scenario illustrates the major accessibility and trust gap in Cameroon's real estate sector.

III. LIMITATIONS OF THE EXISTING SYSTEM

Following our analysis of the current state of price information access in Cameroon's agricultural sector, we have identified several key limitations that affect both farmers and the overall efficiency of the market:

Table 4: limitations of the existing system

Limitation	Consequence
High investment threshold	Ordinary citizens are excluded from real estate ownership and income.
Lack of fractional ownership	Properties cannot be split into affordable shares for small investors.
Manual rent distribution	Delays, inaccuracies, and lack of documentation for rent sharing.
No investment transparency	Informal co-ownership often leads to disputes and mistrust.
Low use of digital tools	Most owners manage rent without software or automation.
Limited investor protection	Investors have no legal proof of ownership or guaranteed income tracking.
High dependency on agents	Many avoid real estate due to high agency fees and lack of trust.

IV. PROBLEMATIC

The major obstacle affecting real estate profitability in Cameroon is the **lack of access to transparent, timely, and localised market price information**. This situation forces farmers to rely on guesswork, informal communication, or physical travel to assess market conditions—often leading to poor sales outcomes.

Therefore, we ask the central question:

“How can we provide farmers with a simple and accessible tool to monitor real-time agricultural product prices, track market trends, and reduce their dependence on unreliable sources and intermediaries?”

V. PROPOSED SOLUTION

Following our analysis of the existing system and its limitations, we propose the development of a web application named “**Green**”, specifically designed for property owners, investors, and tenants in Cameroon. The platform will provide a modern, decentralised approach to property investment and rental income management. Green will enable the following features for its key user groups:

Property Owners

- Tokenise their buildings into digital shares and sell fractions to raise capital.
- Manage tenants and rent collection through a secure digital dashboard.
- Track ownership statistics and real-time investor participation.
- Automate monthly rent distribution through integrated smart contracts.

Investors

- Invest in real estate from as low as 50,000 FCFA by purchasing property tokens.
- Receive monthly rent income based on the number of tokens held.
- Access a personal portfolio dashboard to view earnings, property performance, and token value.
- Benefit from transparent, traceable ownership backed by blockchain technology.

Tenants

- Pay rent online through secure payment gateways (e.g., CAMPAY).
- Access rent receipts and payment history instantly.
- Communicate directly with landlords or building managers via the platform.

VI. CONCLUSION

In this section, we examined the current state of the real estate sector in Cameroon and identified its major shortcomings, including the high cost of investment, manual rent management, and limited access for small investors. Through this research, we developed the concept of **Green**, a tokenised real estate platform that uses digital technology to democratise property investment and improve rental efficiency. By integrating blockchain, smart contracts, and secure payment APIs, **Green** offers a modern, accessible, and transparent solution to both property owners and aspiring investors. This project not only bridges the gap between technology and real estate but also contributes to financial inclusion and economic empowerment in Cameroon.

BOOK III : SPECIFICATION BOOK

Preamble

The Specification book is a document which is established by the customer needing a particular product and the producer (engineer) who is to create the product, which is to be respected for its fullness during the realization of the product. From this, we obtain the major importance of the specification book

Content

INTRODUCTION

- I. CONTEXT AND JUSTIFICATION
- II. OBJECTIVES
- III. EXPRESSION OF NEEDS
- IV. PLANNING OF THE PROJECT
- V. ESTIMATION OF THE PROJECT
- VI. CONSTRAINTS
- VII. DELIVERABLES

CONCLUSION

INTRODUCTION

The specification book sets the foundation for understanding the software project at hand. It provides an overarching summary of the software's purpose, the scope of its functionality and the specific objectives it aims to achieve. It also lays out the intended audience of the software, any assumptions or prerequisites for its use, and the strategic approach to its development. Following this, the specification book delves deeper into the intricate details of the software, covering aspects like functional and non-functional requirements, system limitations and constraints, preliminary user interface design, and data structures of database design. In this document, it will serve as a pivotal roadmap guiding the software development process. We will also include project details such as the team involved, constraints, the budget, deadlines, and deliverables.

I. CONTEXT AND JUSTIFICATION OF STUDIES

a. Context

The real estate sector in Cameroon is experiencing rapid urban population growth, with over 56% of citizens now living in cities such as Douala, Yaoundé, and Bafoussam. Despite this, pathways to property ownership and real estate investment remain restricted by high capital requirements, burdensome legal procedures, and limited access to formal credit and modern investment vehicles. Most properties are bought outright, requiring millions of FCFA, and formal land titling covers only a small fraction of all land—leaving transactions lengthy, costly, and prone to disputes.

Ordinary citizens, especially young professionals, students, and small savers are effectively excluded from building wealth through real estate. Existing co-ownership arrangements lack transparency, structure, and formal automated management. Meanwhile, most rental income is collected manually or through informal digital wallets, with little automation or accountability.

Academic research, policy reports, and market analyses consistently cite these barriers as major contributors to Cameroon's estimated **2.5-million-unit housing deficit** and low rates of property-based wealth creation among most of the population. The situation calls for innovative, inclusive, and technology-driven approaches to democratize property investment and make rental management transparent, traceable, and accessible.

b. Justification

Without a trusted, centralized digital platform dedicated to fractional real estate investment, Cameroon's property market faces persistent systemic problems:

1. **Investment Exclusion:** High costs, opaque paperwork, and minimum investment thresholds block most citizens from participating in property ownership and rental income generation.
2. **Ineffective Management:** Manual or informal methods for co-ownership and rent distribution cause disputes, errors, and a lack of trust.
3. **Lack of Market Transparency:** Investors cannot verify stakes or track income reliably; capital remains illiquid for owners who cannot sell small shares or raise funds without selling full assets.
4. **Legal and Operational Risks:** Reliance on informal co-ownership and manual distribution increases the risk of disputes and undermines confidence in real estate as an investment class.

Studies on fintech and digital real estate markets across Africa show that transparent, accessible, and technologically robust investment platforms can expand property market participation, unlock trapped capital, and automate financial flows, supporting both economic growth and social inclusion.

II. OBJECTIVES OF THE PROJECT

General Objective

The main objective of this project is to design and implement a web-based platform that enables property owners in Cameroon to tokenise real estate and allows individuals to invest in property shares and receive rental income automatically through blockchain smart contracts.

Specific Objectives

To support the general objective, this project will aim to:

- Allow property owners to upload, manage, and tokenise their real estate.
- Enable users to register, log in securely, and manage digital wallets.
- Allow investors to purchase property tokens and track monthly rental earnings.
- Provide smart contract-based automation for rent distribution.
- Integrate local payment APIs for rent collection and investor payouts.
- Offer tenant portals for rent payment, receipts, and property feedback.
- Build a responsive and lightweight platform accessible via desktop and mobile.

III. EXPRESSION OF NEEDS

In this section we are going to outline the specific needs or requirements that the software project is intended to meet. It is a bridge between the problem and the solution detailing the ‘what’ and not the ‘how’. And this need is going to be divided into functional and non-functional needs.

1. Functional Needs

In the context of this project, functional requirements describe what the system should do to meet its objectives. The modules are outlined below:

Administrator (Platform Admin)

- ❖ **User and Role Management:** Create and manage accounts for property owners, investors, and tenants.
- ❖ **Property Management Dashboard:** Approve, verify, and list properties for tokenisation.
- ❖ **Smart Contract Oversight:** Monitor contract deployment, track rent distribution, and ensure contract accuracy.
- ❖ **KYC and Compliance:** Enable user identity verification for secure investment and legal compliance.

For the Property Owner

- ❖ **Property Tokenisation:** Digitally divide real estate into tradable tokens.
- ❖ **Rent Collection Tracking:** Monitor rent payments and automate rent collection via CAMPAY.
- ❖ **Investor Dashboard:** View who owns what % of property, token prices, and engagement.
- ❖ **Update Listings:** Modify property details, upload documents, and images.

For the Investor

- ❖ **Token Purchase and Portfolio View:** Buy tokens and monitor investment performance.

- ❖ **Rent Income Logs:** Track monthly earnings from token ownership.
- ❖ **Transaction History:** View all past purchases and rent payouts.
- ❖ **Sell or Transfer Tokens (Optional Future Feature):** Liquidity option via token resale.

For the Tenant

- ❖ **Rent Payment Interface:** Pay rent online via mobile money or card.
- ❖ **Receipts and History:** Download proof of payment and view past invoices.
- ❖ **Maintenance Requests:** Submit property issues to the owner/manager.

2. Non-Functional Needs

These are essential quality characteristics that ensure the platform is functional, secure, and user-friendly.

- ✓ **Security:** All transactions and data must be encrypted and protected from unauthorised access.
- ✓ **Performance:** Fast-loading interfaces, minimal delays in contract execution and page rendering.
- ✓ **Usability:** A clean, intuitive interface accessible to users with basic digital literacy.
- ✓ **Reliability:** Accurate, consistent data processing and stable uptime for uninterrupted access.
- ✓ **Scalability:** The system must support future expansion, additional properties, and user growth.
- ✓ **Compatibility:** Cross-browser and mobile device compatibility for wide user reach.
- ✓ **Auditability:** Every transaction should be logged, timestamped, and traceable for compliance and trust.

IV. ESTIMATED COST OF THE PROJECT

A. Hardware Resources

Table 5: Hardware Resources

(Sources: 1- <https://www.scribd.com/document/561202966/mercuriale-2024>, 2-mercuriale de prix 2024)

Material	Usage	Qty	Unit Price (FCFA)	Total (FCFA)
Laptop Dell Inspiron 16 (Core i5, 13thgen)	System development, documentation, research	1	587,120	587,120
USB Flash Drive 32GB (Lexar)	File transfer and printing	1	9,860	9,860
Modem Camtel (4G)	Internet access	1	54,275	54,275
HP Officejet Printer 4630	Printing reports and documentation	1	546,250	546,250
Total hardware		4		1,197,505

B. SOFTWARE RESOURCES

Table 6: Software Resources of the project

(<https://www.g2.com/categories/pricing>)

SOFTWARE	USAGE	QUANTITY	PRICE(FCFA)
Microsoft office 365	Used for the creation of the report and power point	1	287,500
Visual studio	Open and free software for the coding the project	1	Free Version
Matterport	API used for the house virtual tour	1	150,000
Visual paradigm	For doing the design of the diagrams to represent the structure and the interaction between the different actors	1	Community version
Microsoft edge	For doing the research on the internet and testing of the application	1	Free Version
Windows 11 (64 bits)HOME	Operating system	1	94,460
Gannt Project	For building a Gannt chart for the planning of the work	1	Free Version
MongoDB	For creating database	1	Free Version
Postman	For testing API's	1	Free Version
Gmail	Collaboration tool ,for fluid communications via mails	1	Free
TOTAL 1			531,960

C. HUMAN RESOURCES

Table 7: Human Resources

(Sources: 1- <https://www.scribd.com/document/561202966/mercuriale-2024>, 2-mercuriale de prix 2024)

	Resources	Quantity	Number of days	COST DAY	PER COST(FCFA) TOTAL
Scrum master	Project manager	1	75	35,000	2,625,000
Development team	Analyst	1	25	28,000	700,000
	Frontend Developer	1	50	32,000	1,600,000
	Backend Developer	1	70	32,000	2,240,000
	Tester	1	5	12,000	60,000
	Designer	1	5	22,000	110,000
		Total			7,335,000

D. GLOBAL ESTIMATION

Table 8: Total Project Estimation Cost

Type	Price
Hardware	1,197,505FCFA
Software and subscription	531,960FCFA
Human Resources	7,335,000FCFA
Overall total	9,064,465FCFA
Margin of error = (15% of overall total)	1,359,670 FCFA
Final total Overall total + margin of error)	10,424,135FCFA

Ten millions four hundreds and twenty four thousand one hundred and thirty five FCFA

V. PROJECT PLANNING

a) Chronogram of activities

Table 9: chronogram of activities

PHASE	OBJECTIVE	OUTPUT	DURATION	PERIOD
INSERTION PHASE	Study of the enterprise	Insertion Phase	14 days	14 th July to 27 th of July
SPECIFICATION BOOK	Specification of the user needs	Specification Book	10 days	28 th July to 7 th August
ANALYSIS	Capture of needs Use case and textual description Modelling	Analysis Book	20 days	8 th August to 26 th August
CONCEPTION	Preliminary conception and Detailed conception	Conception book	15 days	27 th September to 10 th September
REALIZATION	Implementation Unitary test Integration Test Development, Deployment, Component diagrams	Realization book	14 days	11 th September to 24 th September
TEST OF FUNCTIONALITIES	Testing of the software and debugging	Test of functionalities	5 days	25 th September to 29 th September
INSTALLATION AND USER GUIDE	Documenting software	User Guide	1 day	30 th September

b) GANTT CHART

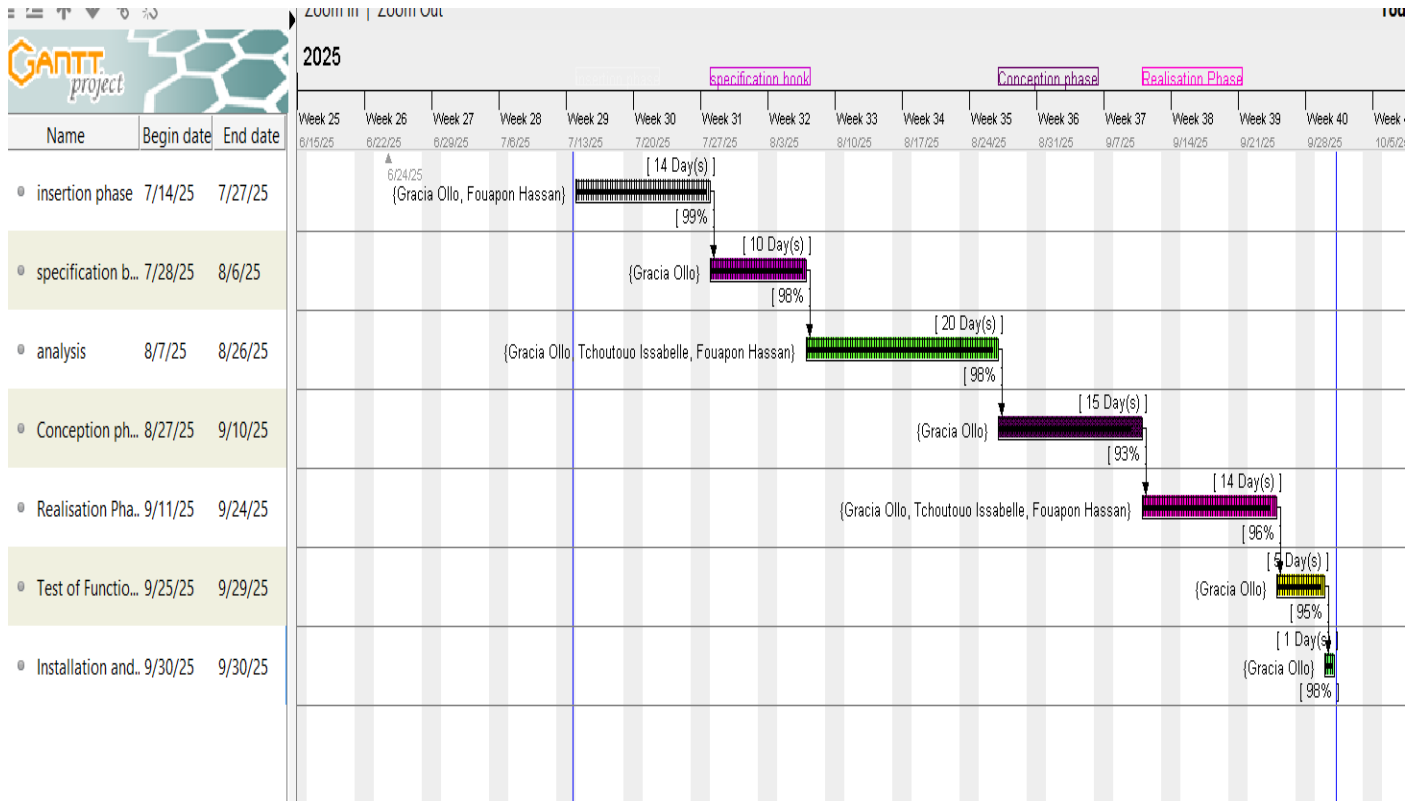


Figure 3: Gantt Chart

VI. PROJECT CONSTRAINTS

Constraints refer not only to obstacles but also to the requirements given to us by our supervisor for the work we need to deliver. We can classify them as follows:

- ❖ Cost constraints: The implementation of our project will require expenses in terms of human resources, materials, and software, with a total cost of **10,424,135 FCFA**.
- ❖ Time constraints: This project needs to be completed within a period of 75 days, from 14th july to 31st september 2025.
- ❖ Quality constraints: Security and robustness

VII. LIST OF PARTICIPANTS AND DELIVERABLES

a) List of participants

Table 10 : list of participants

NAME	TITLE	ROLE
GRACIA OLLO CLEMENCIA MBISSICK	Software engineer student at AICS-Cameroon	Analyst and developer
Mrs. TCHOUTOUO ISABELLE	Lecturer at AICS-Cameroon	Academic supervisor

b) Deliverables

We are supposed to submit a report at the end of this project that includes;

- A CD: The CD will contain the setup program of the developed application, along with its source code. The user guide may also be included on the CD, although it will also be part of the deliverables mentioned later;
- A report: The report will contain a comprehensive analysis of the system we have developed, including diagrams related to the modelling language that was used.
- A prototype/mock-up: This will be used to simulate the expected output and provide a visual representation of the application's design and functionality.

CONCLUSION

The specification book helps us to site the different needs we need to implement our application for generator monitoring; we saw list of participants and deliverables for our project. In time allocated for this section, we were able to accomplish the different task, we will move directly to the next phase, which is the analysis phase. In the analysis phase, we will study the existing system in detail and model our system with a modelling language and process.

BOOK III : ANALYSIS BOOK

Preamble

After specification book, we have the Analysis phase, which permits us to represent a detailed analysis of the limitations identified in our context, and our solution, through a software development process and modelling language

Content overview

INTRODUCTION

- I. COMPARATIVE STUDIES BETWEEN UML AND MERISE
- II. COMPARATIVE STUDIES OF UNIFIED PROCESS
- III. JUSTIFICATION METHOD OF ANALYSIS
- IV. MODELLING OF THE PROPOSED SOLUTION

CONCLUSION

INTRODUCTION

The analysis book permits us to examine in an explicit way the existing system, its limitations and how we can remedy them. We will also describe in detail the modelling language known as UML (Unified Modelling Language) which is coupled with Two Tract Unified Process (2TUP) to form a method and its justification why we decided to use it in preference of another. Then we will dive directly into the modelling of the proposed solution consisting of diagrams that meets the requirements of the functional needs.

I. COMPARATIVE STUDY OF UML AND MERISE

1. MODELLING WITH MERISE (2.5)

MERISE stands for “Méthode d’Etude et de Réalisation Informatique pour des Systèmes d’Entreprise”. Although it is prescriptive to some extent, MERISE permits the participation of end users and senior management as well as data processing professionals in its decision cycle. MERISE is a method for designing, developing and carrying out IT projects. The goal of this method is to achieve the design of an information system. The MERISE method is based on the separation of data and processing to be carried out in several conceptual and physical models. The essentials of the approach lie in its three cycles: the decision cycle, the life cycle and the abstraction cycle, which cover data and process elements equally. The separation of data and processing ensures longevity in model. Indeed, the arrangement of data does not have to be often overhauled, while treatments are more frequently.

2. MODELLING WITH UML (2.5)

The unified modelling language (UML) is a general purpose, developmental modelling language in the field of software engineering that was intended to provide a standard way to visualize the design of a system. The Unified Modelling Language (UML) was standardised in January 1997 by the Object Management Group (OMG), which is an American association created in 1989 and aims to promote and standardize the object model in all its forms. In 2005, UML was also published by the international organization for standardization (ISO) as an approved ISO standard. UML since 2015 is in its version 2.5. This version consists of fourteen diagrams classified into structural and behavioural diagram

a. Structural Diagrams

Structural diagrams represent the static components of a system; they emphasize on what should be in the system we are modelling. They include:

- ❖ Class diagram;
- ❖ Object diagram;
- ❖ Package diagram;
- ❖ Composite structural;
- ❖ Deployment diagram;
- ❖ Component diagram;
- ❖ Profile diagram

b. Behavioural Diagrams

The behavioural diagrams capture the dynamic state of a system; they emphasize on what should happen in the system we are modelling. They are:

- ❖ Use case diagram;
- ❖ Activity diagram;
- ❖ State machine diagram;
- ❖ Sequence diagram;
- ❖ Communication diagram;
- ❖ Interaction overview diagram;
- ❖ Timing diagram

II. UML 2.5 diagrams overview

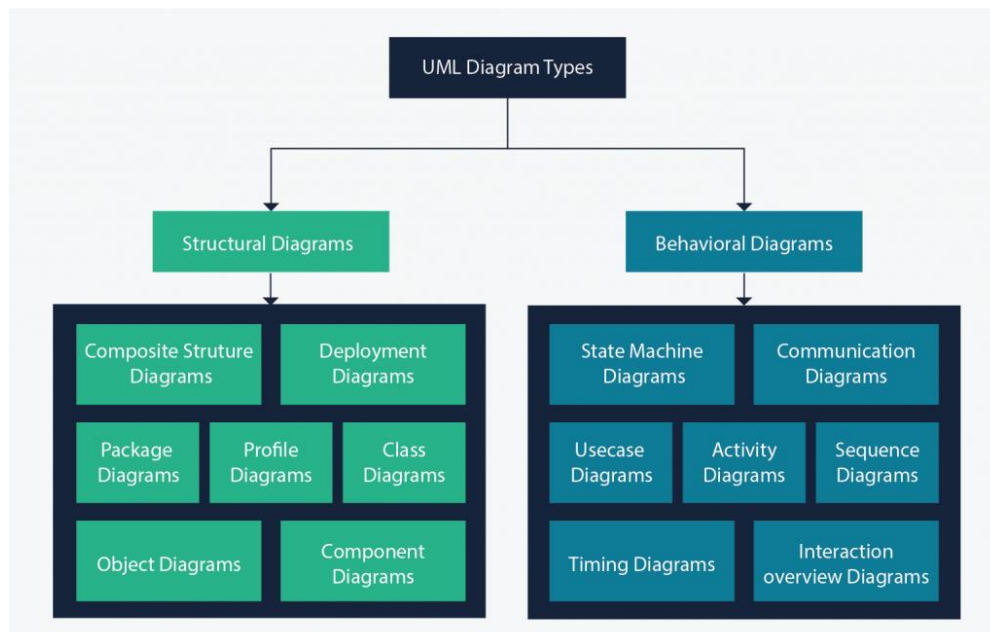


Figure 4: UML diagram overview

(source: <https://creately.com/blog/diagrams/uml-diagram-types-examples/>)

It is important to note that UML is a modelling language and not a method or procedure. Hence, to give it an approach, we need to associate UML to a Unified process (UP) to give our conception a methodology to follow. A unified process is a generic method for developing software. This implies it is necessary to adapt the UP to the context of the project, team domain and or the organization. We will use the Two Tracks Unified Process (2TUP) throughout our project to implement our solution.

Table 11: Difference between UML and MERISE

MERISE	UML
It stands for Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprises	Unified Modelling Language
MERISE is a systemic method of analysis and design of information systems. That is, it uses a systems approach.	UML is however not a method but rather an object modelling language to which it is necessary to associate an approach to make it a method. This is the case with the 2TUP method; RUP and XP
MERISE proposes to consider the real system from two points of view: - A static view (data) - A dynamic view (treatments). That is, with the MERISE method, we have a separate study of the data and the treatment	UML offers a different approach from that of MERISE in that it combines data and processing. Because with UML, centralizing the data of a type and the associated processing makes it possible to limit the maintenance points in the code and facilitates access to information in the event of software development. In addition, UML describes the dynamics of the information system as a set of operations attached to the objects of the system.
Rational	Object

III. COMPARATIVE STUDY OF UNIFIED PROCESSES

A process can be defined as a partially sequence of steps that permits us to obtain software systems or evolution of an existing one. The main objective of software development is the production of quality software that response to the needs of the users during a particular time and at a particular cost.

a. A Unified Process

A unified Process is a process of development of software constructed on UML; it is iterative, incremental, cantered on architecture, driven by use cases and requirements.

Iteration is distinct sequence of activities with a basic plan and evaluation criterion that produces an internal or external output. Either the content of an iteration is improved, or users evaluate the evolution of the system.

An increment is the difference between two released products at the end of two iterations. Each iteration that the group can integrate the technical environment to develop a final product and give users the possibility of having tangible results.

Centred on architecture the different models derived during the establishment of system must be reliable and coherent.

Driven by use case and requirements enables the clear definition of a users' needs and priorities respectively thereby minimizing the risk of project failure.

b. The Two Track Unified Processing (2TUP)

2TUP is a unified process, which is belt on UML and has as objective to bring solution to constraints of functional and technical changes imposed on information systems by strengthening controls on development capacities. It proposes a Y-sharped development life cycle that separates the functional aspect from the technical aspects, and the merging of these two forms the implementation aspect. 2TUP distinguishes therefore two branches: the functional and technical branches, the combination of the result of these two branches forms the third: the realization branch – where we realize our system. The diagram bellow illustrates the branches of 2TUP.

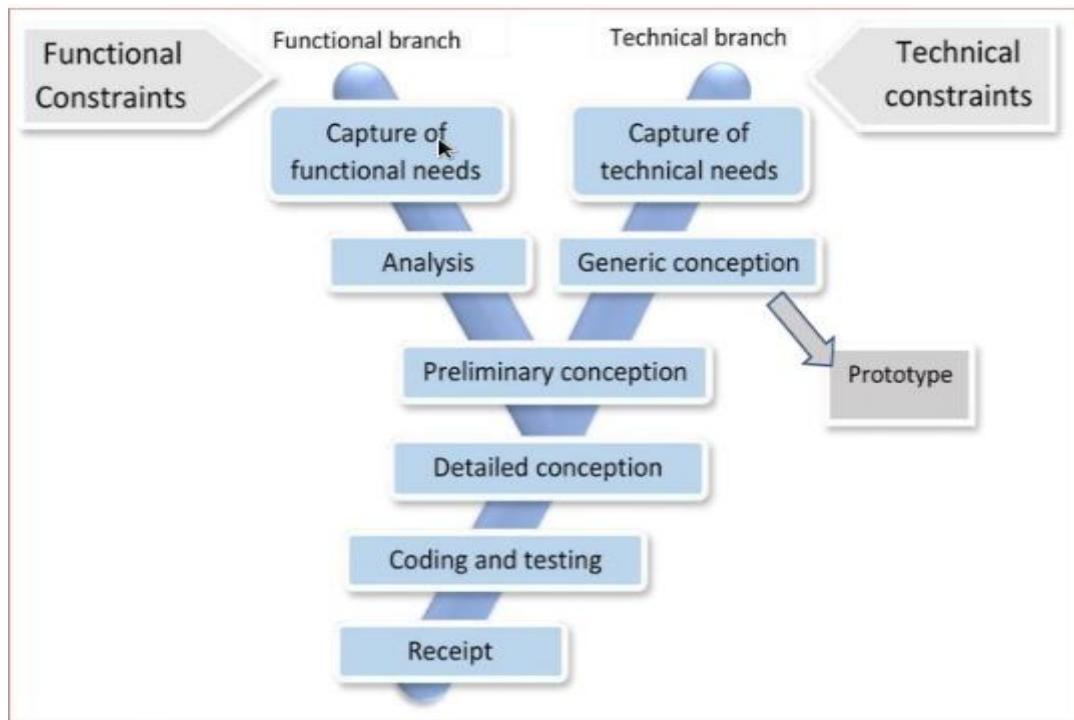


Figure 5: 2TUP diagram

(Source: <https://www.mysciencework.com/omniscience/pervasive-mobile-healthcare-system-basedon-cloud-computing>)

a. The left branch (Functional branch)

It captures the functional needs of a system. This ensures the production of software that meets the needs/requirements of the user. The analysis here consists of studying precisely the functional specification to obtain an idea of what the system is going to realize, and its result does not depend on any technology

b. The right branch (Technical branch)

The technical branch enumerates the technical needs and proposes a generic design validated by a prototype. The technical needs include constraints and choices related to the conception of the system, the tools and equipment as well as the integration constraint with the existing system condition. The different diagrams are shown in the table below.

c. The middle branch (Realisation branch)

In this branch, we study the preliminary conception, detailed conception, and documentation of the system. The realization branch supports the following:

Preliminary conception: This is the most sensitive step of 2TUP as it is the confluence of the functional and technical branch. It is completed when the deployment model, the operating model, the logical model, interphases and the software configuration model are defined.

Detailed conception: This is the detailed design of each feature of the system.

Coding and testing: This are the phase where we program the designed features and test the coded features.

The recipe: Also known as, the deliverables is the validation phase of the functions of the developed system.

III. JUSTIFICATION OF METHOD OF ANALYSIS

The reason why we chose UML modelling language and the software development process 2TUP instead of many others that exist, include: UML is the current standard for programming in an object-oriented language. For this reason, it is widely understood and well known making it easy for a new programmer to join the project and be productive from the very first day.

UML diagrams allow teams to virtualise how a project is or will be working, and they can be used in any field, not just software engineering. The diagrams will allow teams to virtualize together how a system or a process will work or did work. It can provide new ideas for how teams must collaborate to achieve the goal of the workflow process. 2TUP is centered around the creation and maintenance of a model, rather than the production of mountain documents.

2TUP is user oriented as it permits the development of software that responds to the needs of the users through the study of the user needs. 2TUP is iterative and incremental, hence it enables the project team to produce refined amelioration if necessary and easily integrate it in the already existing system.

2TUP by permitting the project team identify and test the key functionalities of the system limits the risk related to building the system.

IV. MODELLING OF THE PROPOSED SOLUTION

A. Capture of Functional Needs

The first step of the left (functional) branch of Two Track Unified Process (2TUP) is the capture of the functional needs. At this step, we capture the intended behavior of the system that maybe express as services, tasks or functions the system is required to perform.

1. USE CASE DIAGRAM

a) Definition

Use case diagram shows the functionalities of a system, their interdependencies and how they relate with actors of the system. A use case is a specification of behaviour. The main objectives of the use case diagram are:

- ❖ Provide a high-level view of the system.
- ❖ Identify the functions of the system.

Use case diagrams are completed with a textual description of each use case that is intended to define the use case in greater details

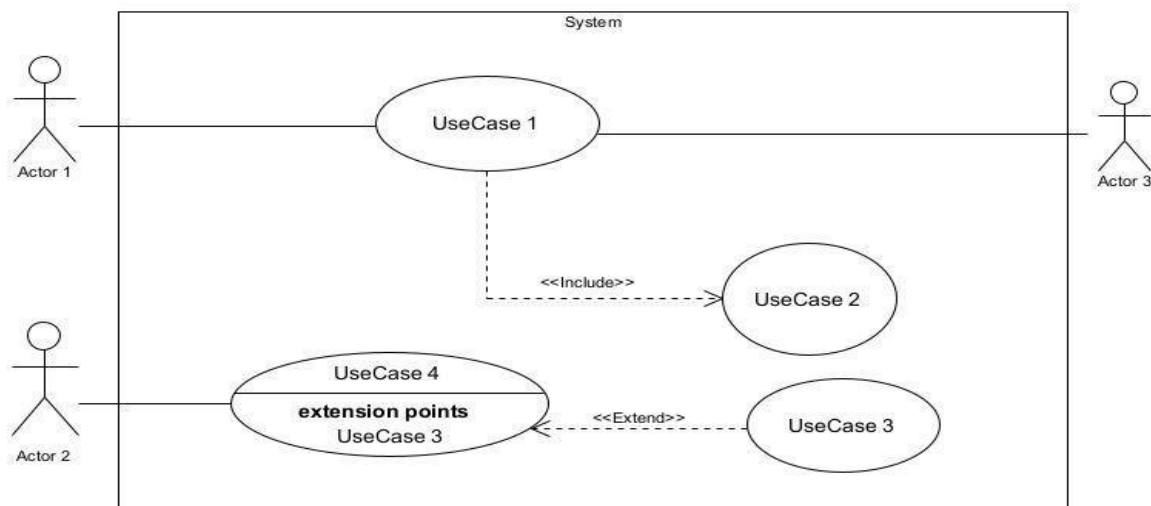
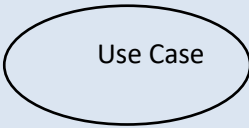
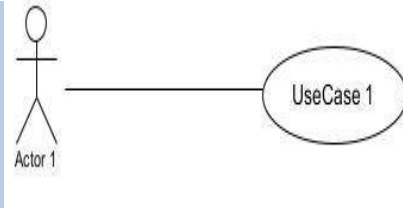
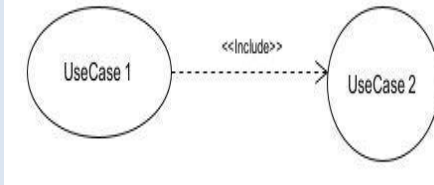
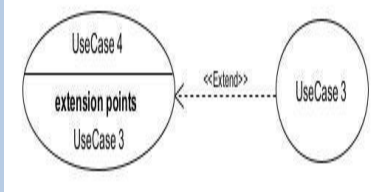
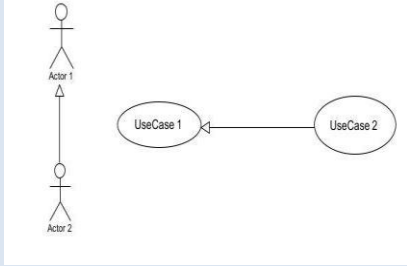
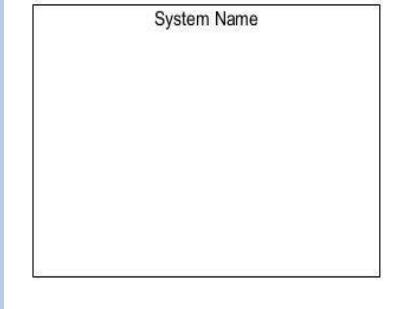


Figure 6: Use Case Diagram of formalism

b) Elements of a use case diagram

Table 12: use case Diagram Component

Elements	Notation	Description
Actors		Represents an entity that directly interacts with the system. The actor is what performs the different possible actions of the system.
Use case		A use case represents a functionality of the system. It is an action that can be performed by an actor.
Association		it indicates that an actor takes part in a use Case.
Include		An inclusion denotes that an included action must be performed before the including action can be performed.
Extend		An extension denotes that an extending action may be performed while an extended action is being performed.

Generalization		This shows that an actor or a use case is a kind of another abstract or concrete actors can be defined and later specialized using generalization relationship.
System		It is a container of use cases which interact with external actors

c) The Actors of Our System

Primary Actors

Table 13: Actors of our system

Actor	Role
Visitor	View crop details
Farmer	He can add his products i.e. crops, fertilisers etc. , view market prices and vote for prices
Buyer	View products, contact farmers, make payments
Administrator	Manages product listings, updates the database, manage prices, manage accounts and monitors app activity.

Secondary Actors

Table 14: secondary actors of our system

Actor	Role
Nasdaq API	Provides the international market prices
CAMPAY API	Help in making payments

d) Textual description of authenticate

Table 15: Textual Description of Authenticate

Title	Authenticate
Summary	This use case allows users to access the platform.
Actors	All users
Date	28/05/2025
Manager	Gracia Ollo
Version	1.0
Preconditions	The user has n account on the platform
Trigger	The user clicks on the login button
Nominal Scenario	▪ The user enters the platform URL ▪ The system displays the login form ▪ The user enters login credentials ▪ System checks input validity ▪ System sends data to the database ▪ DB returns authentication result ▪ System grant access to workspace
Alternative Scenario	4a. Wrong credentials 4b. Error message 6a. No match in DB
Exceptional Scenario	3a. More than 3 failed attempts 3b. System delays for 15s
Post condition (Success)	Access is granted
Post condition (Failure)	Access is denied

Make Payments

Table 16 : Textual Description of make payment

TITLE	MAKE PAYMENT
Summary	This use case allows buyers to make payment buying products (crops or fertilizer)
Actors	Buyer Secondary actor payment API
Date	28/05/2025
Stakeholders	Gracia Ollo
Version	1.0
Precondition(s)	The actor has authenticated The user has clicked on the product page
Triggers	The user clicks on the (buy) button
Nominal Scenario	1 -The actor clicks on the buy button 2 -The buying form displays 3 -The user fills and submits the form. 4 -The actor clicks on the payment button. 5 -The system displays the payment form. 6 -The actor fills and submits the form. 7 -The system verifies conformity of the form fields 8 -The system sends the request to the payment API. 9 -The API returns result of the query. 10 -The system displays a success message to the actor.
Alternative Scenarios	-At step 6a of the nominal scenario, the users' account balance is low.

	-6b the system displays an error message and sends him back to step 5
Exceptional Scenario	6a The users' data connection stopped 6b The front end system send an error message
Postcondition of success	The payment is done
Postcondition for failure	The user did not make the payment

General Use Case Diagram

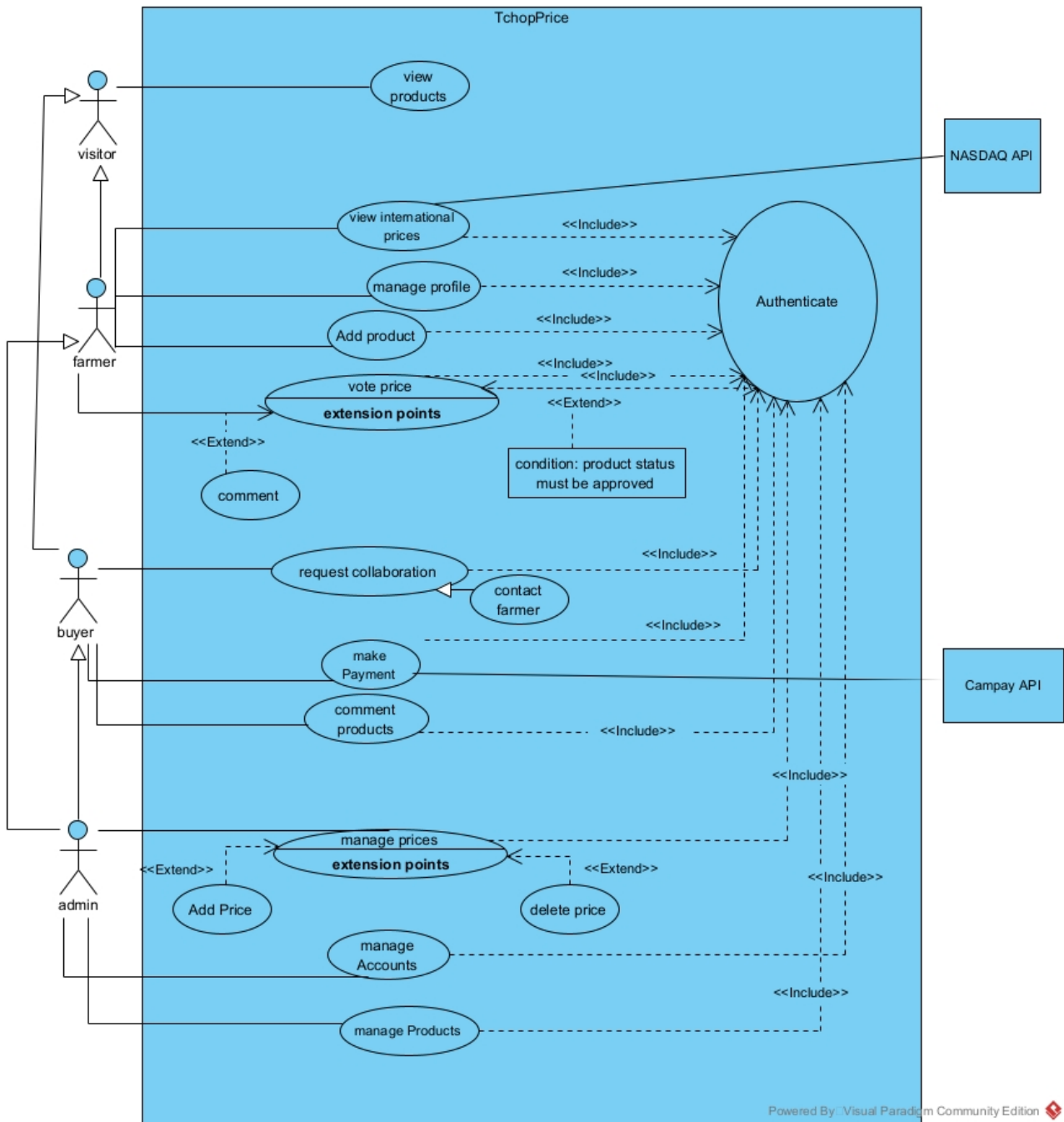


Figure 7: General use case diagram

Specific use case Manage Products

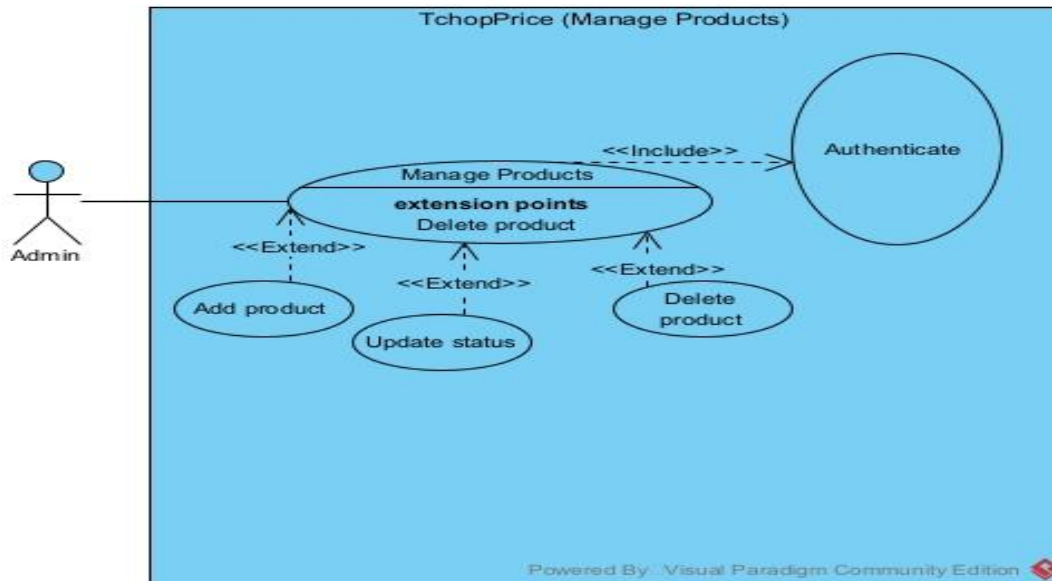


Figure 8: specific use case of manage products

Specific use case of make transaction

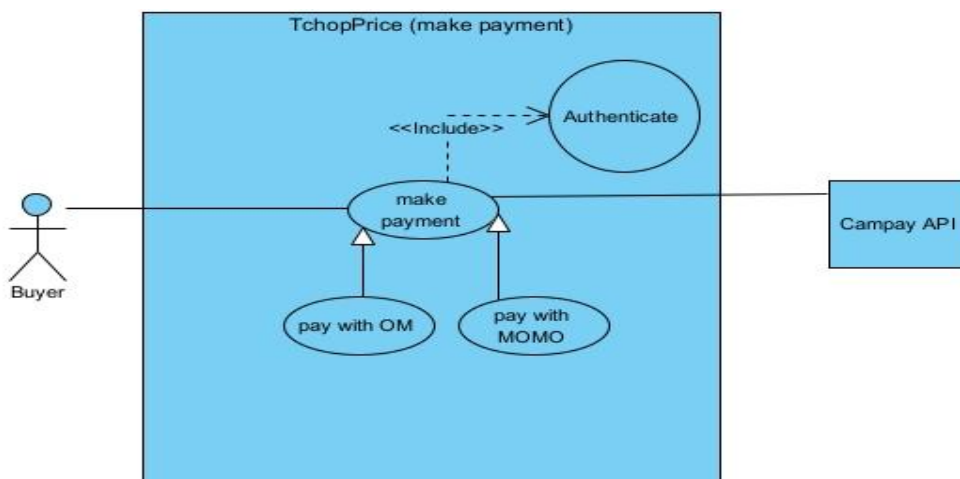


Figure 9: make Payment use case diagram

2. Communication Diagram

a) Definitions:

It is a diagram which is used to show the relationship between the actors of a system, both the sequence and the communication diagrams represent the same information but differently. Instead of showing the flow of message. It depicts the architecture of the object residing in the system as it is based on object-oriented programming.

b) Formalism

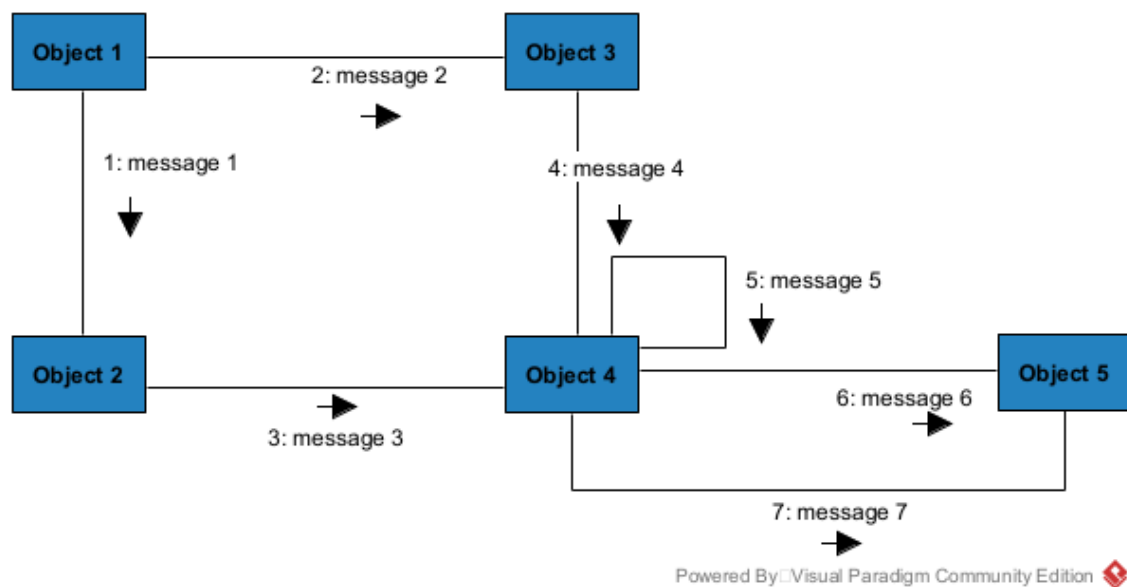
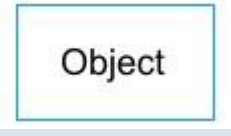


Figure 10: Communication Diagram Formalism

Table 17: Communication Diagram Component

Element	Representation	Description
Object		An actor represents an individual participant in the interaction conversation.
link		It initiates an association it connects two objects together for them to communicate.
Actor		A role plays by an entity that interacts with the subjects.
message		Defines a particular communication between lifelines in an interaction.

PIOL Communication Diagram for Authentication

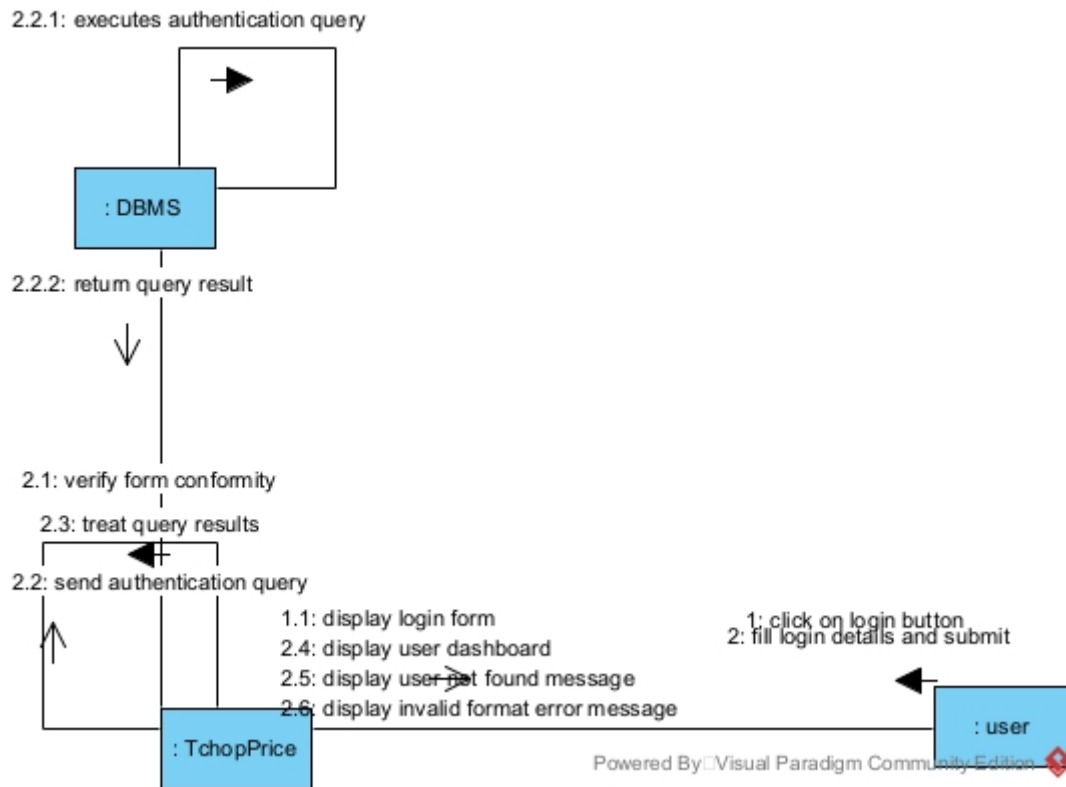


Figure11: Communication Diagram of authentication

Communication diagram Vote price

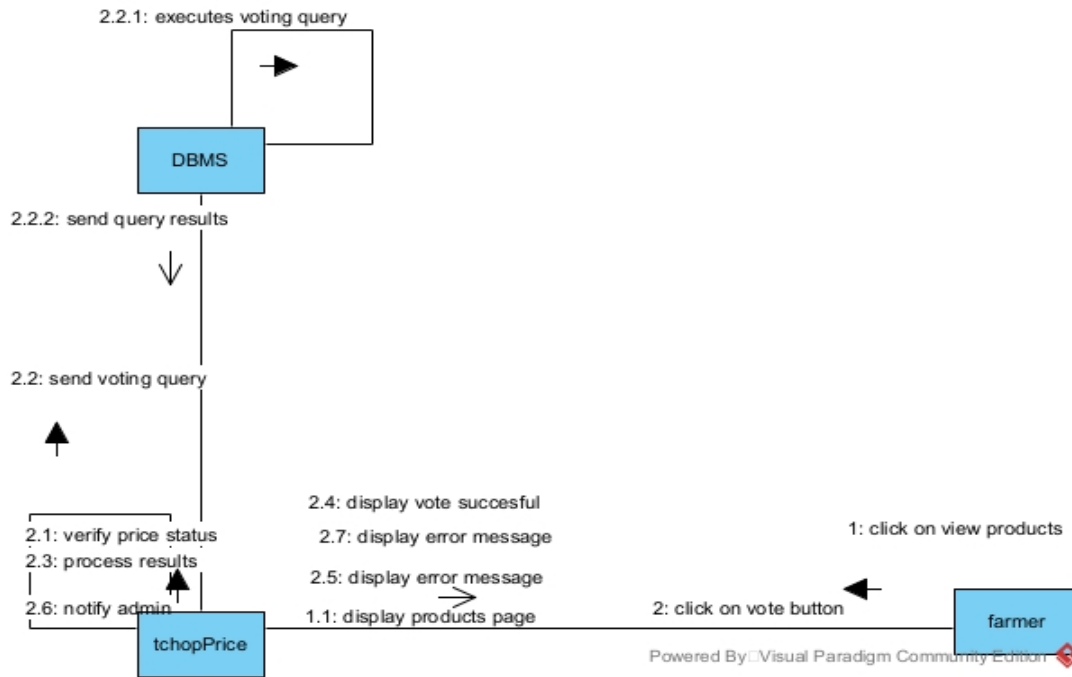


Figure 12: Vote communication diagram

3. Sequence Diagram:

a. Definition:

A Sequence diagram describes interactions among classes in terms of an exchange of messages over time. They are also called event diagrams. A Sequence diagram is a good way to visualize and validate various runtime scenarios. These can help to predict how a system will behave and to discover responsibilities a class may need to have in the process of modelling a new system.

b. Formalism:

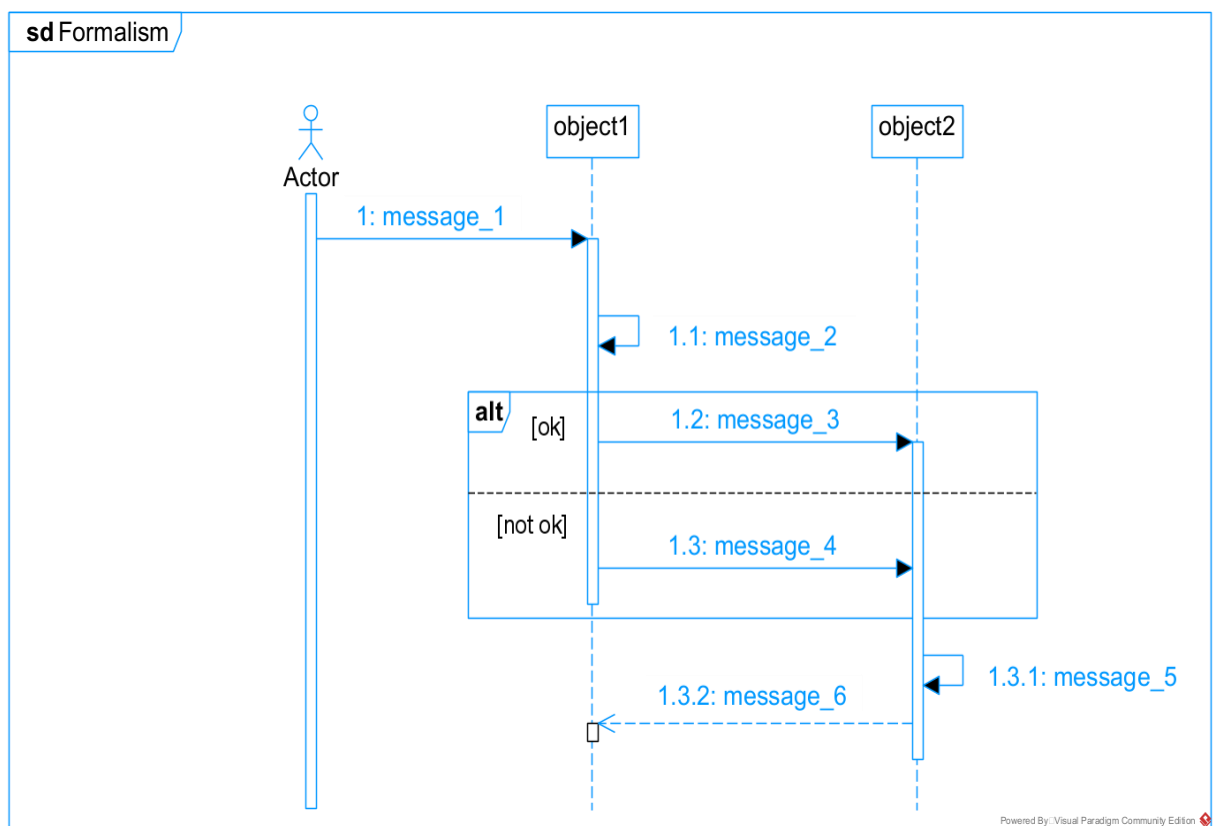
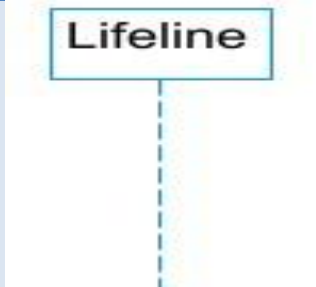
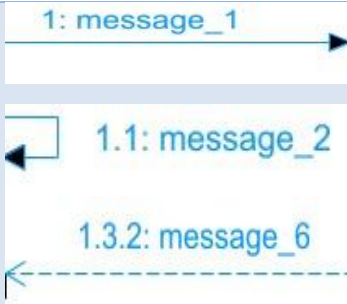
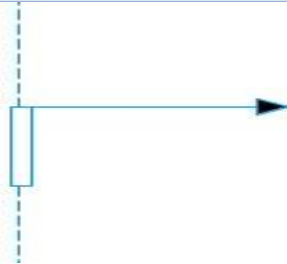


Figure 13: sequence Diagram Formalism

Table 18: Sequence Diagram Components

Element	Representation	Description
Lifeline		An individual participant in a sequence diagram, it is position at the top of the diagram.
Combined fragment		It represents a choice of behaviour in which at most one operand will be chosen.
Messages		These are arrows which shows the direction of message flow. We have the synchronous, the asynchronous and the self messages.
Activation		It describes the time in which an operation is performed by an element.

Authentication Sequence Diagram

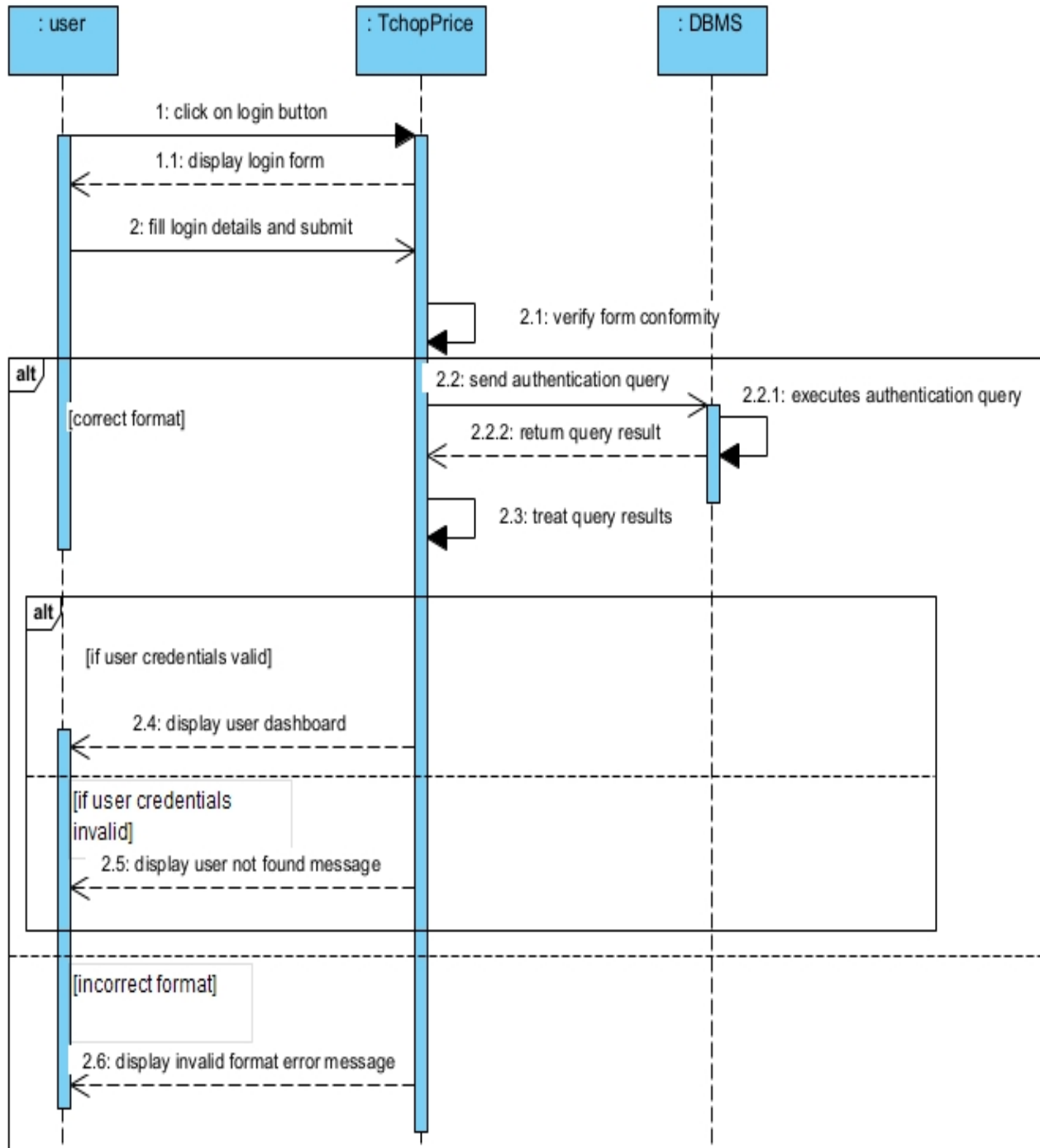


Figure 14: Sequence Diagram of authentication

Sequence Diagram of vote price

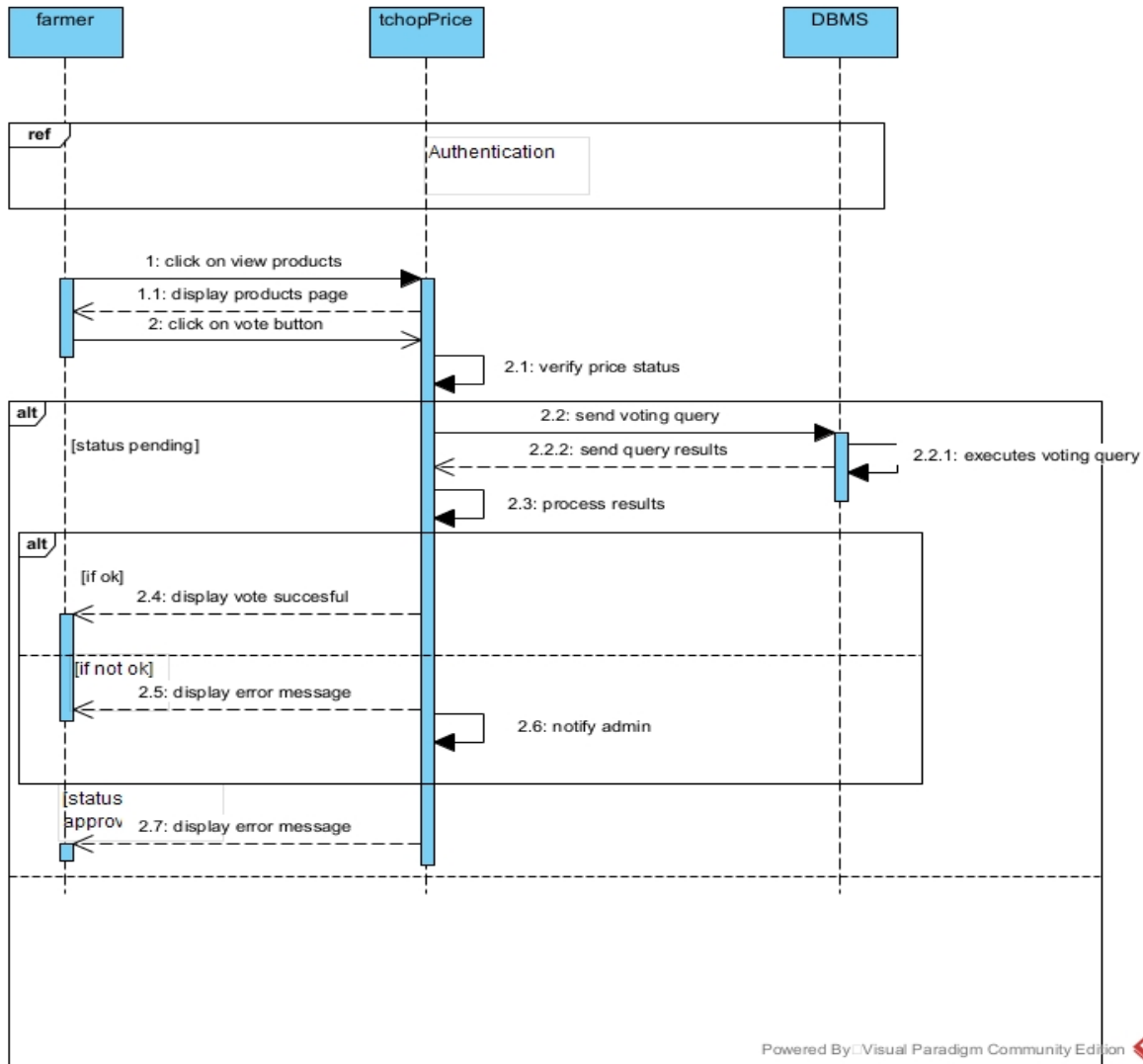


Figure 15: Sequence diagram to vote price

4. Activity Diagram

a) Definition:

An activity diagram is a graphical representation of work-flows that show the steps needed in the realization of a process; showing the details from a start point to an end point through all decisions and actions that can possible be performed. Activity diagrams are intended to model both the computational and organizational process. They flow can be sequential, branched or concurrent. Below is an activity diagram formalism.

b) Formalism

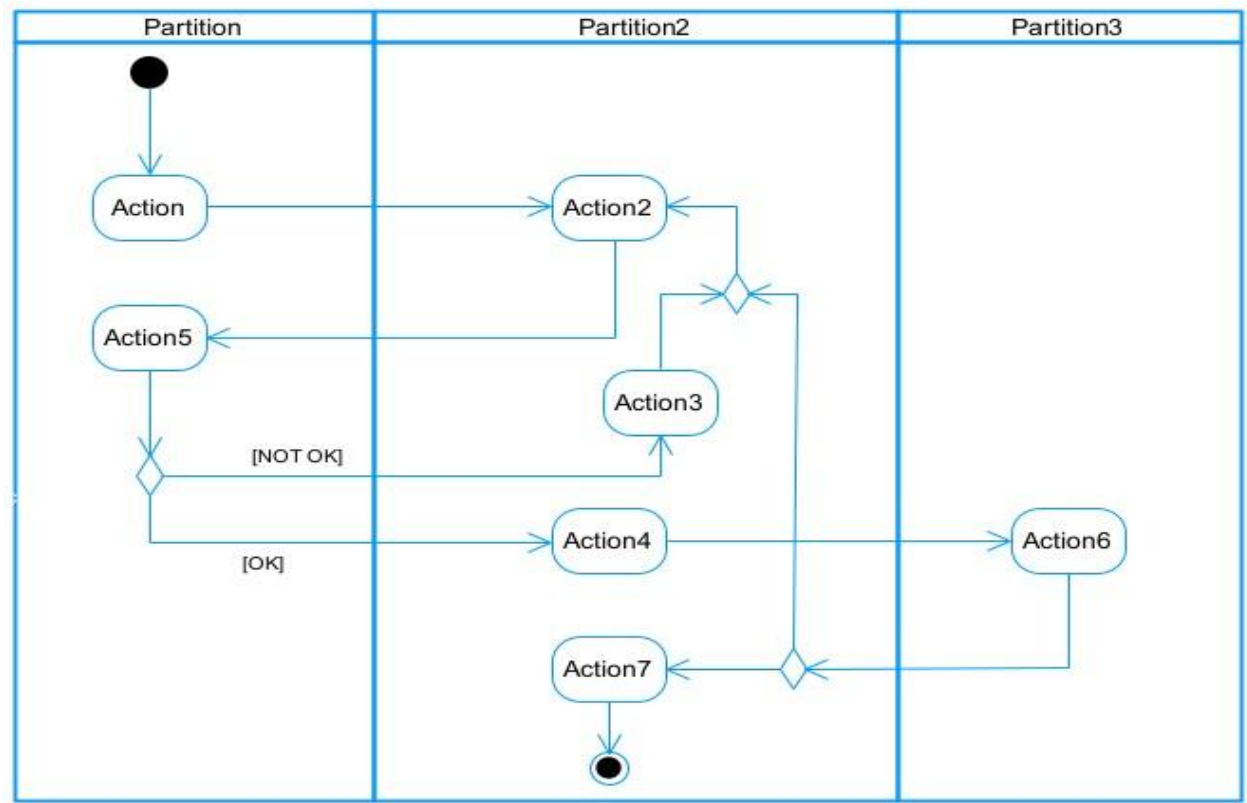
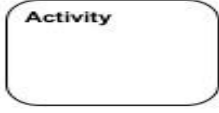
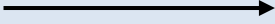
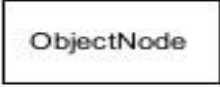
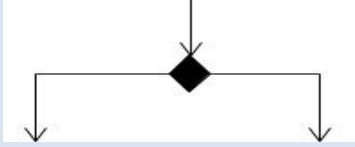
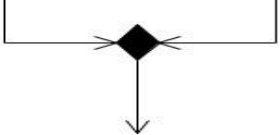
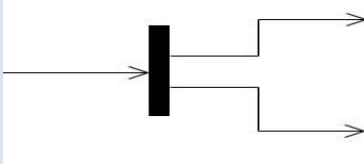
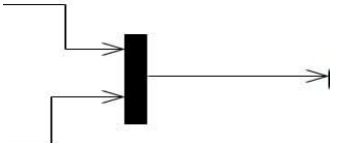
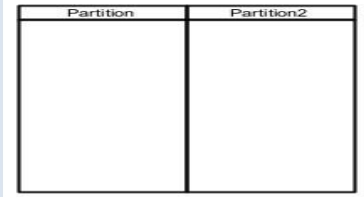


Figure 16:Activity Diagram formalism

c) Activity Diagram Elements

Table 19: Components of an activity diagram

Elements	Diagrammatic Representation	Description
Activity		Use to represent a set of actions.
Action		Represent a task to be performed.
Activity edge		A directed connection between two activity nodes through which tokens may flow.
Initial node		Shows the beginning of an activity or set of actions.
Final node		Stops all controls and object flows in an activity.
Object node		Represents an object connected to a series of object flows.
Decision node		Represents a test condition that splits an incoming activity edge into opposite outgoing activity edges.
Merge node		Reunite different decision paths created using a decision node.

Fork node		Slits behaviour into parallel or concurrent flows of activities (or actions)
Join node		Unites a set of parallel or concurrent flows of activities or actions.
Swimlane and partition		A way of grouping activities performed by the same actor in an activity diagram or to group actions in the same thread.

d) Activity Diagram to authenticate

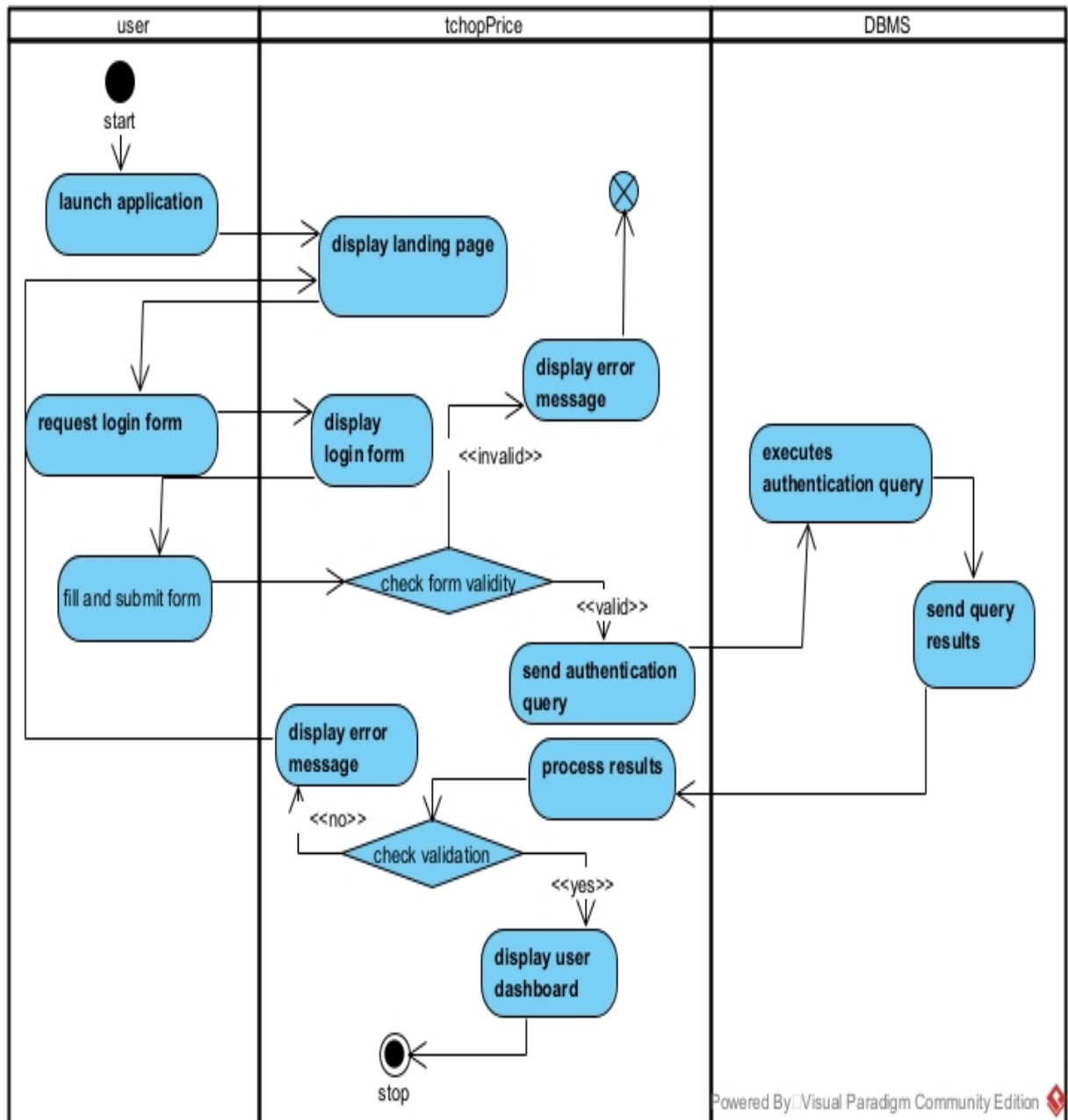


Figure 17: Activity diagram of authentication

e) Activity Diagram to vote Price

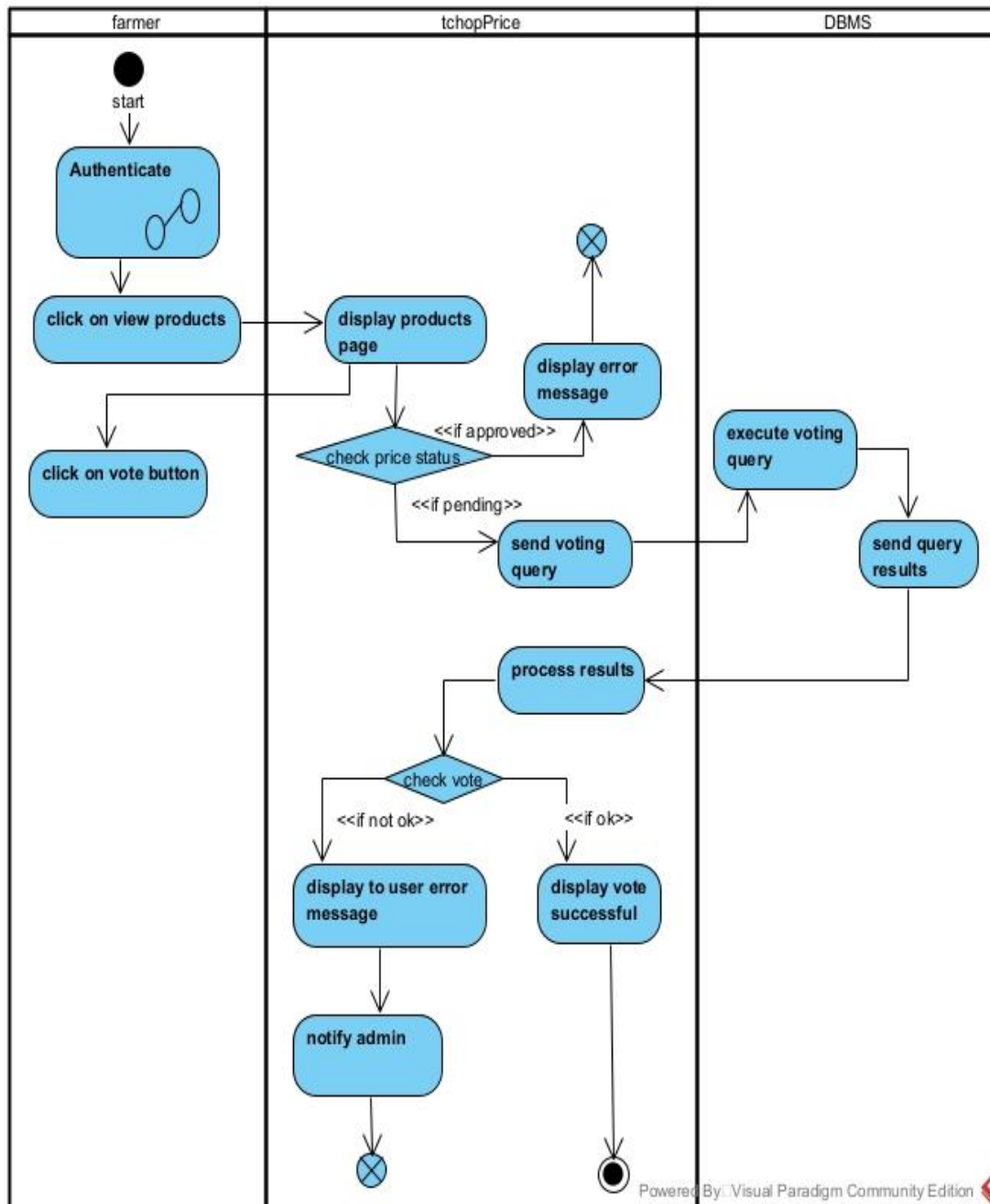


Figure 16:vote price activity diagram

CONCLUSION

In the analysis phase, we chose a software development process and modelling language, after which we presented the existing system, its limitation and our proposed solutions. We went forth explaining the functional need of our system, we saw the use case diagram which shows the relationship between the actors and use case (the action the actor can perform on the system), we saw the activity diagram which shows the workflow of our system, and lastly the State machine diagram which shows the behaviour of a single object in response to an event. We will now move to the conception phase in which we will present the other two branches of the 2TUP which are the Technical and the realization branch of our system.

BOOK IV : CONCEPTION BOOK

Preamble

The conception phase will permit us to present in an orderly manner the components necessary for the good functioning of our software and the architecture used for the proposed solution. It bridges the gap between the analysis phase and the realisation phase.

Content

INTRODUCTION

- I. TECHNICAL BRANCH
 - A. Generic Design
 - B. Capture Of Technical Needs
- II. IMPLIMENTATION BRANCH
 - A. Preliminary Design
 - B. Detailed Design
- III. RELATED UML DIAGRAMS
 - i. CLASS DIAGRAM
 - ii. STATE MACHINE DIAGRAM
 - iii. PACKAGE DIAGRAM

CONCLUSION

INTRODUCTION

The conceptual phase will describe in detail the necessary specifications, features and operations that will satisfy the functioning requirements of the proposed system as modelled in the analysis phase. This phase is meant to identify and consider essential components (hardware /or software), structure (network capabilities), processes and procedures for the system to accomplish its objectives. We will look at some diagrams such as the state machine diagram, package diagram and class diagram.

I. TECHNICAL BRANCH

A. Generic Design

a. Hardware diagram of the system

The hardware diagram simply shows how the system components of our system are deployed ; it shows the positioning of each device into it right proportion

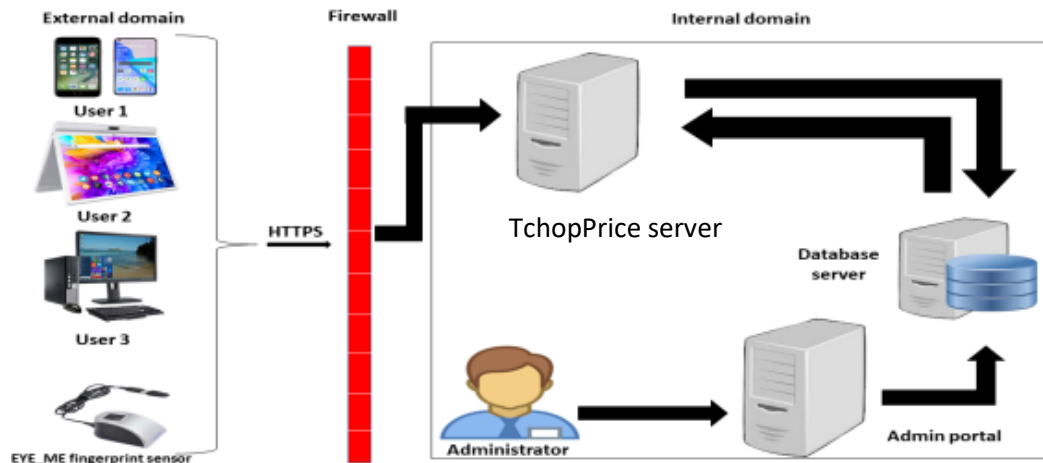


Figure 18: Hardware Diagram

b. High Level Architecture of the System

The high-level architecture diagram provides an overview of the entire system, identifying the main components that would be developed for the product and their interfaces.

B. Capture of Technical needs

a) Physical Architecture

The design of the DBMS depends on its architecture. An n-tier architecture partitions on the whole system into related but separated n modules, which can be independently modified, altered, changed or replace. A large amount of data on web servers, personal computers (pc) and others are link with networks with the help of basic client or server architecture.

Within the scope of our project, we made use of the n-tier architecture This architecture separate it tiers from each other based upon the user and the manipulated data in the database. Each layer has a well-defined communication interface, and the evolution of the layer is independent of the other. The n-tier of our system is made up of:

- The user tier, which is also known as our presentation interphase.
- Application Tier, which represents our webserver.
- The data tier, which represents our DBMS server.

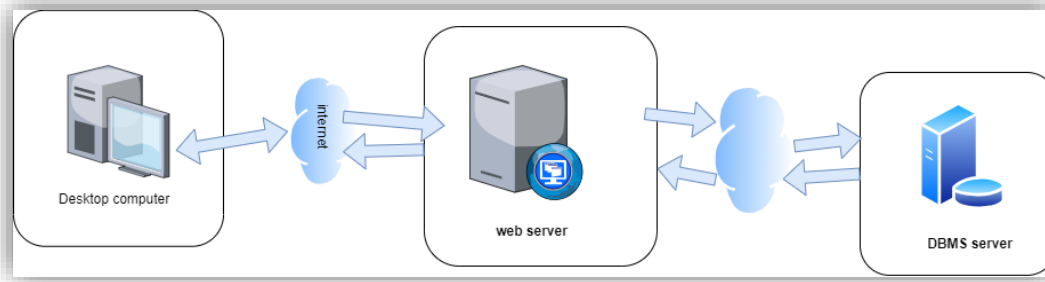


Figure 19: Physical Architecture

b) Logical Architecture

Model View controller or MVC as it is popularly called, is a software design pattern for developing application. A model view controller pattern is made up of the following three parts.

- **Model:** The lowest level of the pattern, which is responsible for maintaining data.
- **View:** This is responsible for displaying all or a portion of data to the user.
- **Controller :** It handles software codes that controls the interactions between the model and the view.

MVC is popular as it isolates the application logic from the user interface and supports separation of concerns. Here the controller receives all requests for the application then works with the model to prepare data needed by the view. The view then uses the data prepared by the controller to produce a final response. The MVC can be represented as follows.

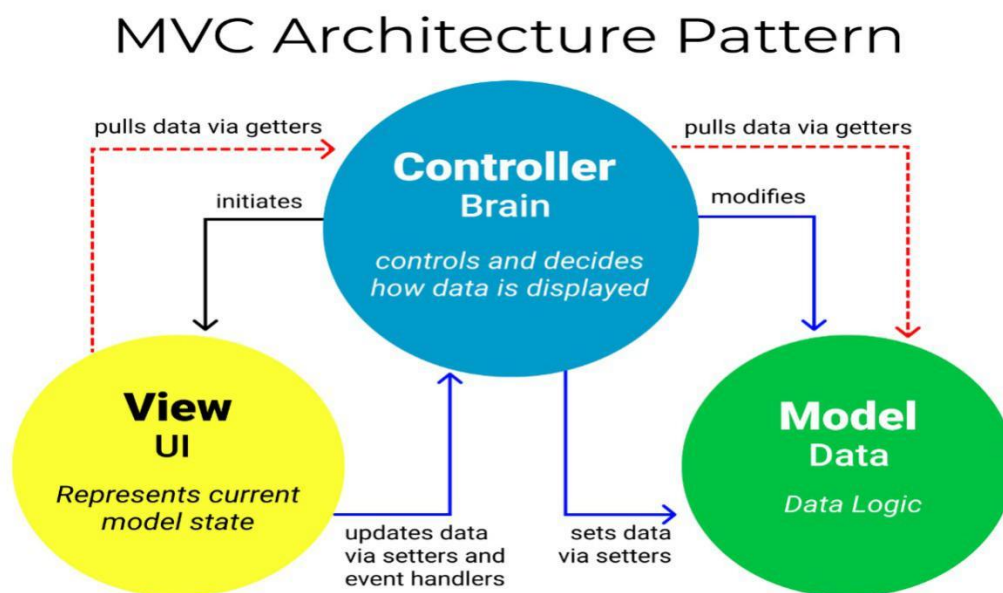


Figure 20: The MVC architecture

Source: <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvcarchitecture-and-frameworks-explained/>

I. RELATED UML DIAGRAMS

A. Class Diagram:

a) Definition:

A class diagram is a static diagram. It represents the static view of an application. class diagram is not only used for visualising, describing and documenting different aspect of the system but also for constructing executable code of the software application. Class diagram describes the attribute and operation of a class and constraints imposed on the system. Its purpose is to model the static view of an application

b) Formalism:

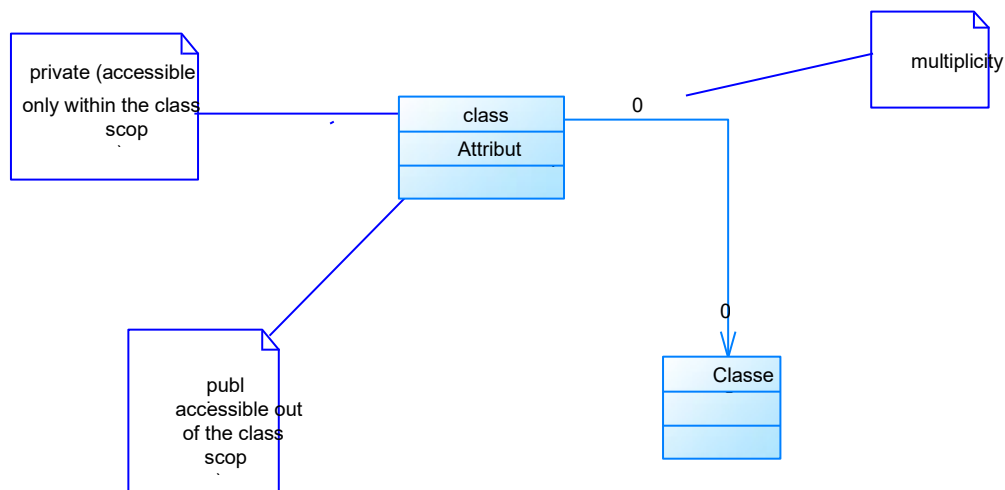
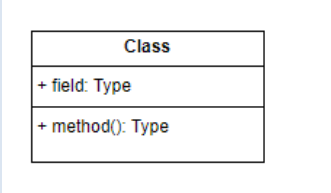
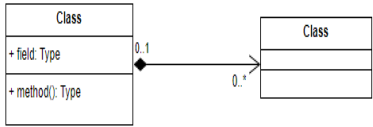
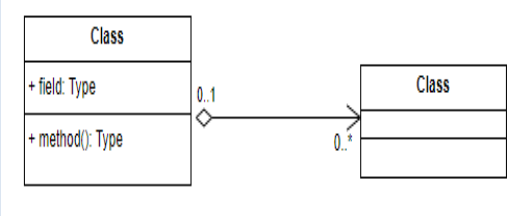
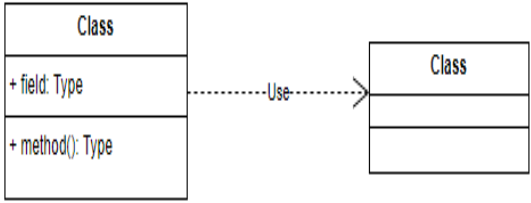
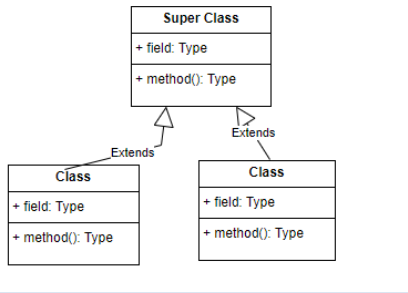


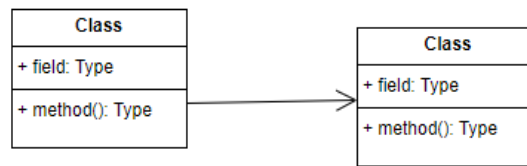
Figure 21: Class Diagram formalism

a) Components of a Class Diagram

Table 20: Components of a class diagram

Element	Representation	Description
Class		A class is an element that defines the attributes and behaviours that an object can generate
Composition		If a parent of a composite is deleted, usually, all its parts are deleted with it.
Aggregation		If the parent of the aggregate is deleted, usually the children are not deleted.
Dependency		It existed between two classes, if one changes it may cause the change in the order, but the other way around
Generalization		It a relationship between a whole thing (called superclass) and a more specific thing (called subclass)

Association



It is a general type of relationship between elements, it may include cardinality, roles etc.

CLASS DIAGRAM

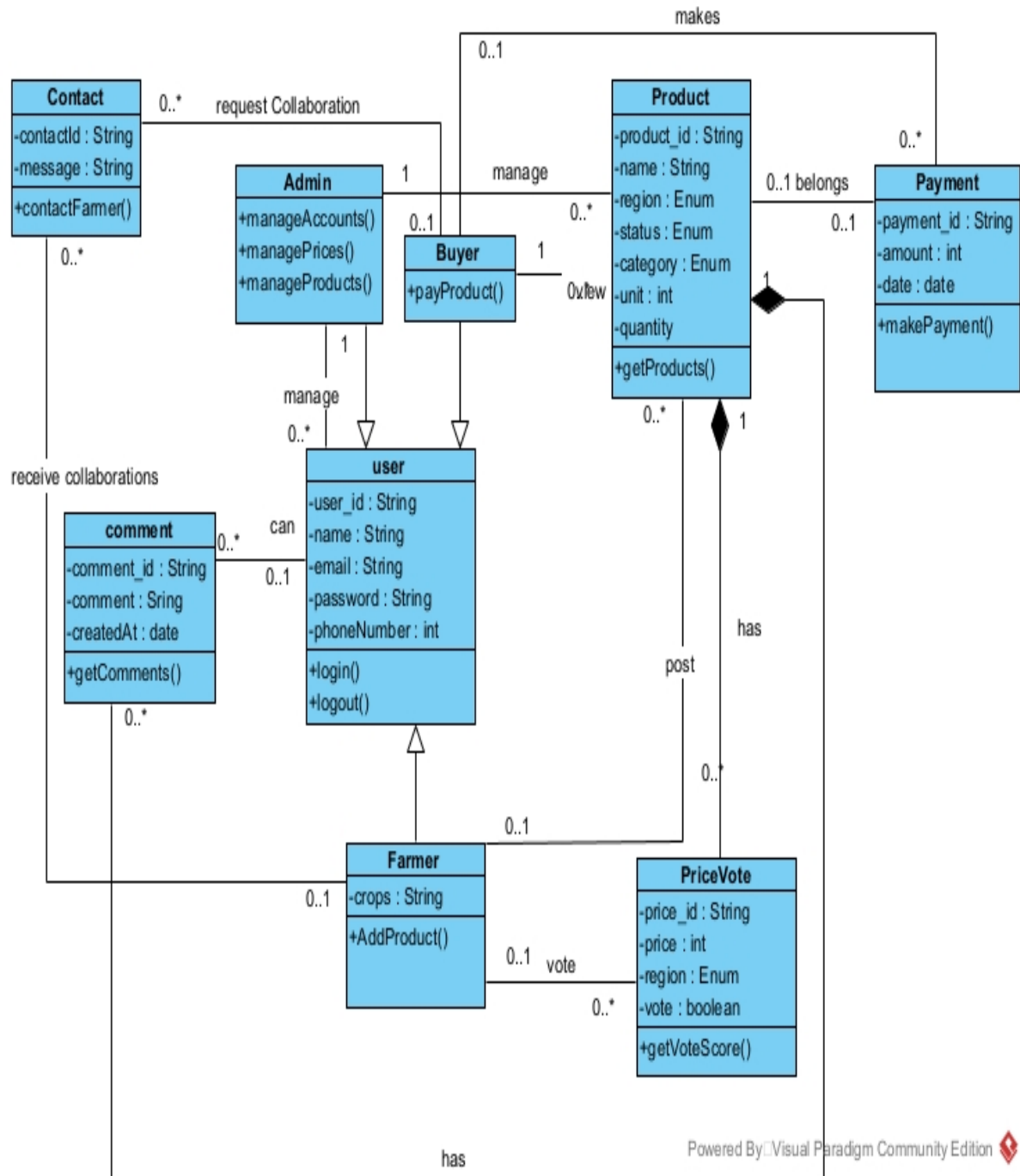


Figure 22: Class diagram

a) Relationship between Classes

Table 21: Relationship between classes

Relationship	Action Performed
R1	A User can make zero or many Comments, and each comment is written by exactly one user.
R2	Admin, Buyer, and Farmer are all specialized types of User via inheritance, with specific roles and operations.
R3	An Admin manages zero or more Products, and each product is managed by exactly one admin.
R4	A Buyer can make one or more Payments, and each payment is made by exactly one buyer.
R5	A Farmer can post zero or more Products, and each product belongs to one farmer.
R6	A Payment belongs to one or more Products, and each product can be linked to zero or one payments.
R7	A User can cast zero or more Price Votes, and each vote is made by one user for one product.
R8	A User can request collaborations via Contact, and each collaboration can be sent to one or more users.
R9	A Product has zero or more Price Votes, but each Price Vote belongs to exactly one product.
R10	A Comment can be left on collaborations or user activities, and each Comment is tied to exactly one user.

State Machine Diagram

a) Definition

A state machine diagram describes the behaviour of a single object in response to a series of events in a system. Also known as the state machine diagram, it models the dynamic flow of control from the state of a particular object within a system.

b) Formalism:

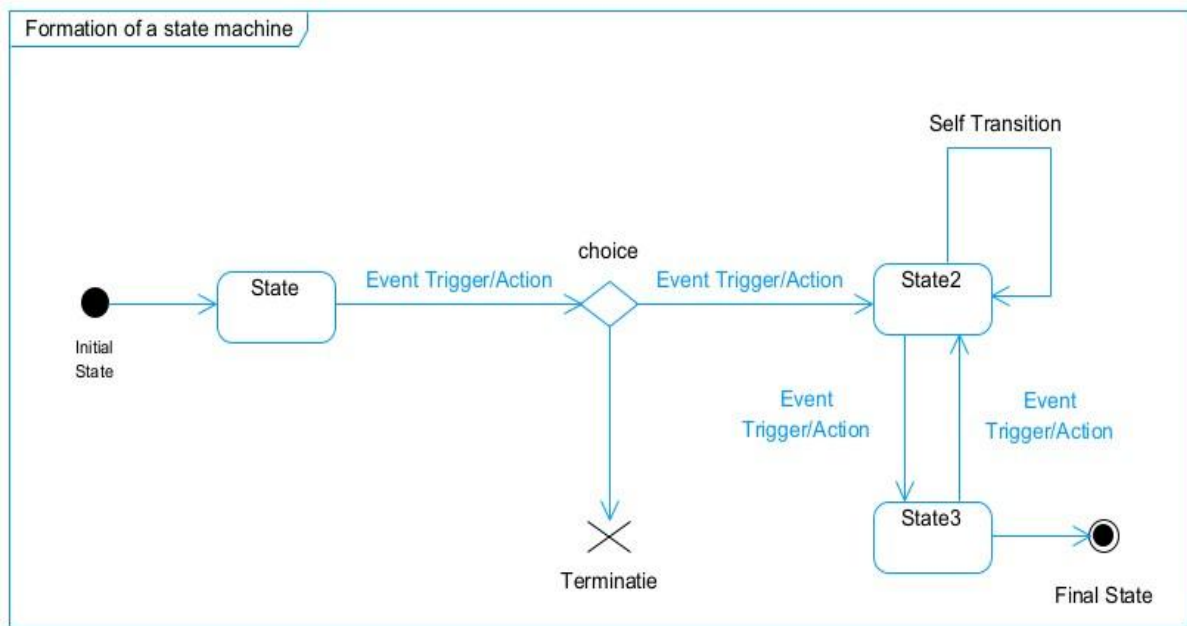
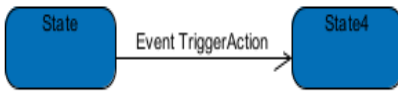
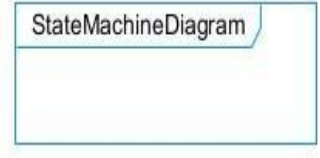


Figure 23: Formalism of a state machine diagram

State Machine Diagram Elements

Table 22: State machine Diagram Components

Component	Diagram Relationship	Description
State		Models a situation during which a certain invariant condition holds.
First (Initial) State		It represents a default vertex, that is, a source for a single transaction to the default or composite state.
Final State		A state specifying that the enclosing region is complete.
Transition		A direction relation between a source and a target vertex.
Choice pseudo-State		A diamond symbol that indicates a dynamic condition with branched potential results
Terminate		Implies that the execution of a state by means of it context is terminated.
Diagram Overview		A placeholder for the linked states in a state machine diagram.

State Machine Diagram

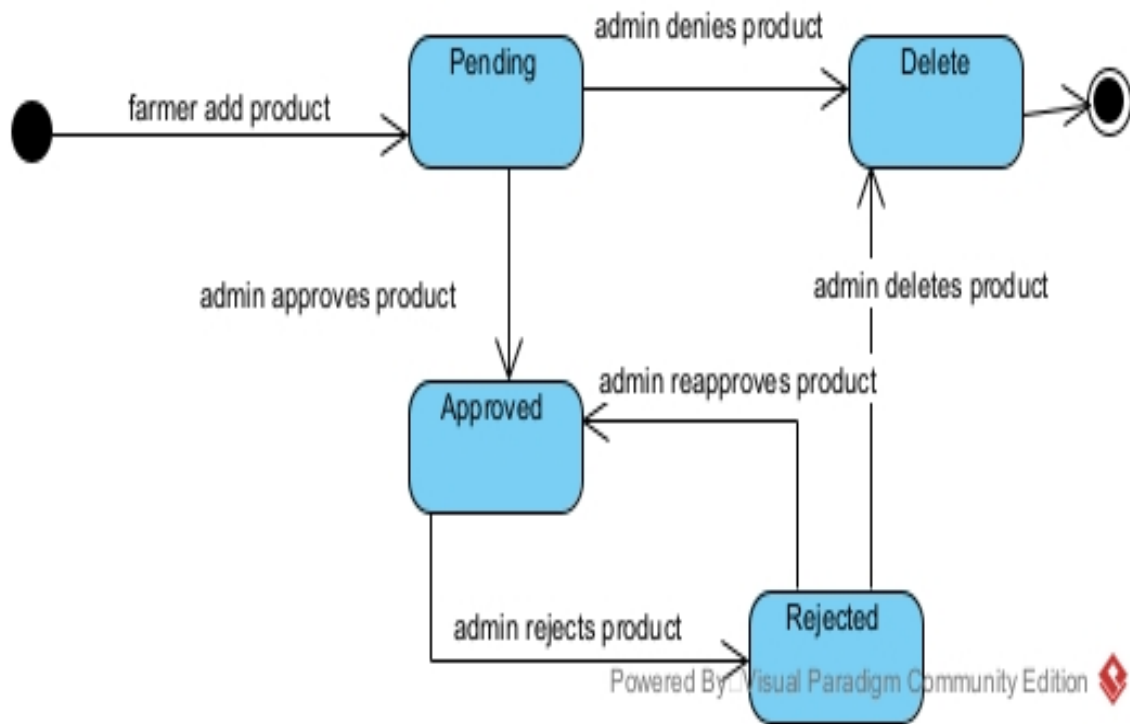


Figure 24: State machine Diagram of

State machine diagram of rent property

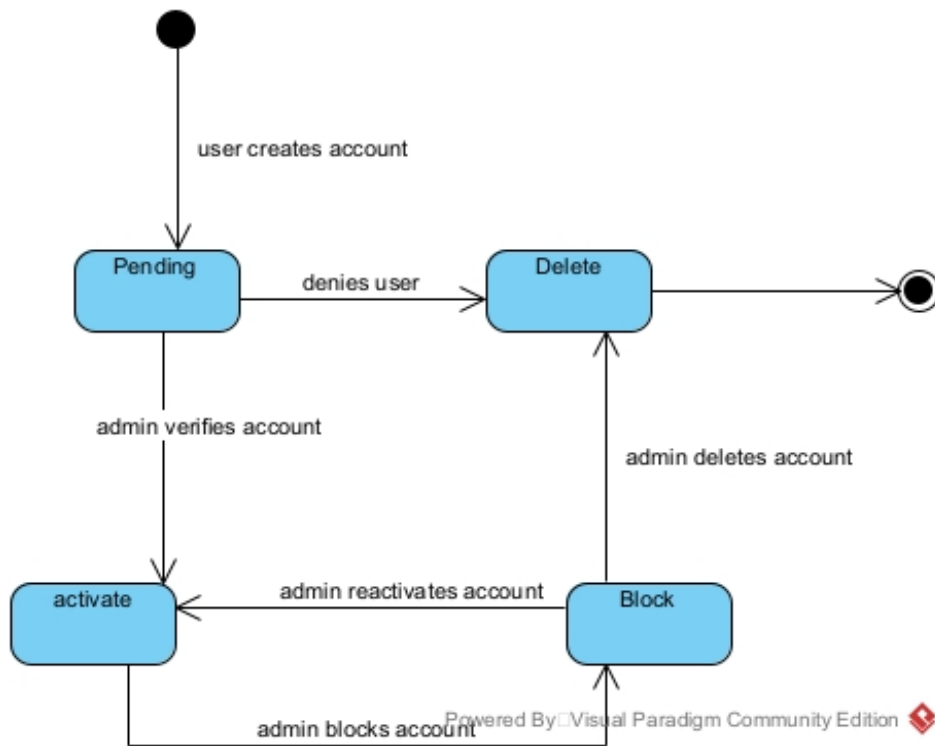


Figure 25: State machine diagram of rent property

Package Diagram:

a) Definition:

This is a structural diagram used to show the organization and arrangement of various model elements in the form of packages. A package diagram is the grouping of related uml elements such as classes, diagrams or eve other packages.

b) Formalism:

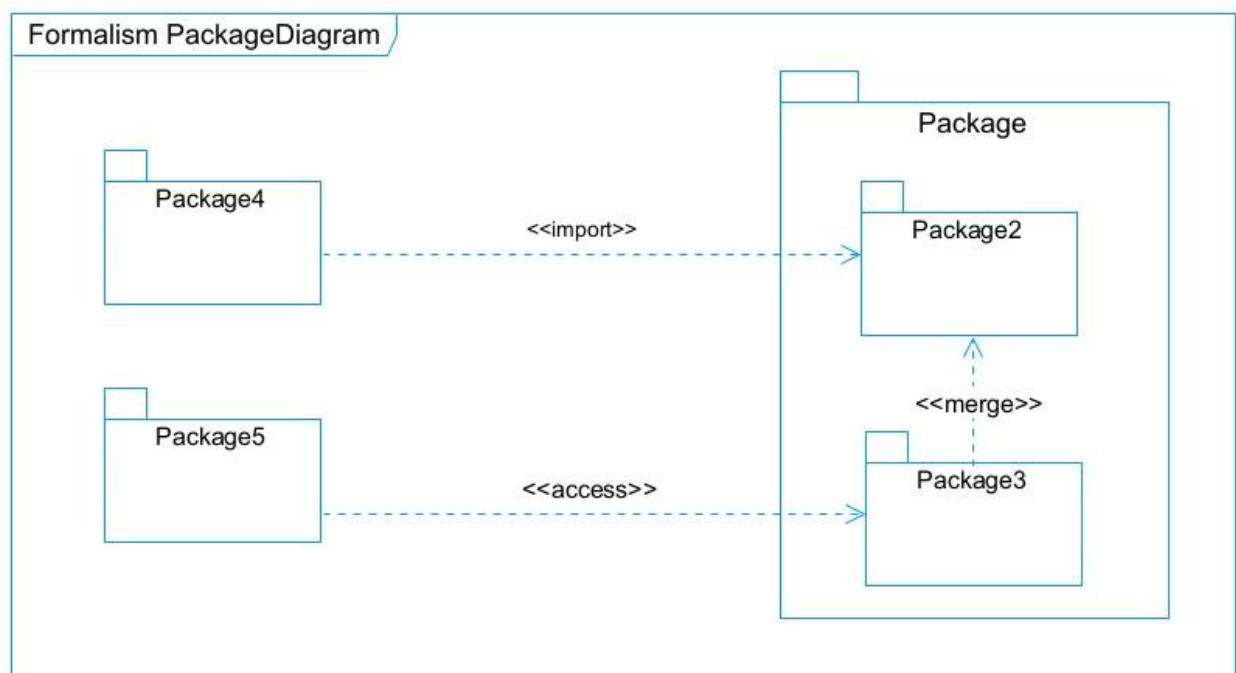
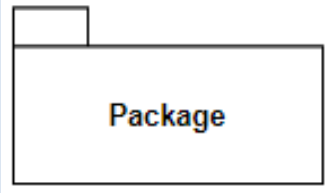
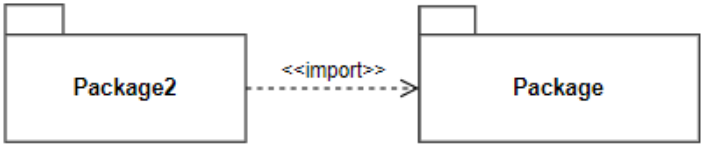
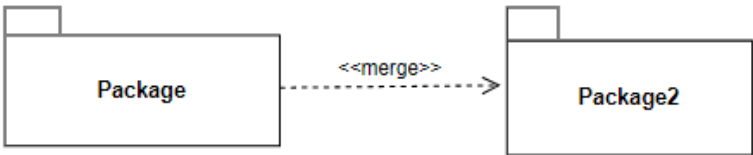


Figure 26:Package Diagram Formalism

c) Components of Package Diagram

Table 23:Package Diagram Components

Name	Diagrammatic Representation	Description
Package		A package is a namespace use to group related elements; it is a mechanism used to group elements into a better structure in a system.
Package import		A relationship Indicate that, functionality has been imported from one package to another.
Package access		A relationship Indicates that one package requires assistance from the function of another package.
Package merge		It is a relationship which shows that, the functionality of two packages are combines to a single function

d) System Package Diagram

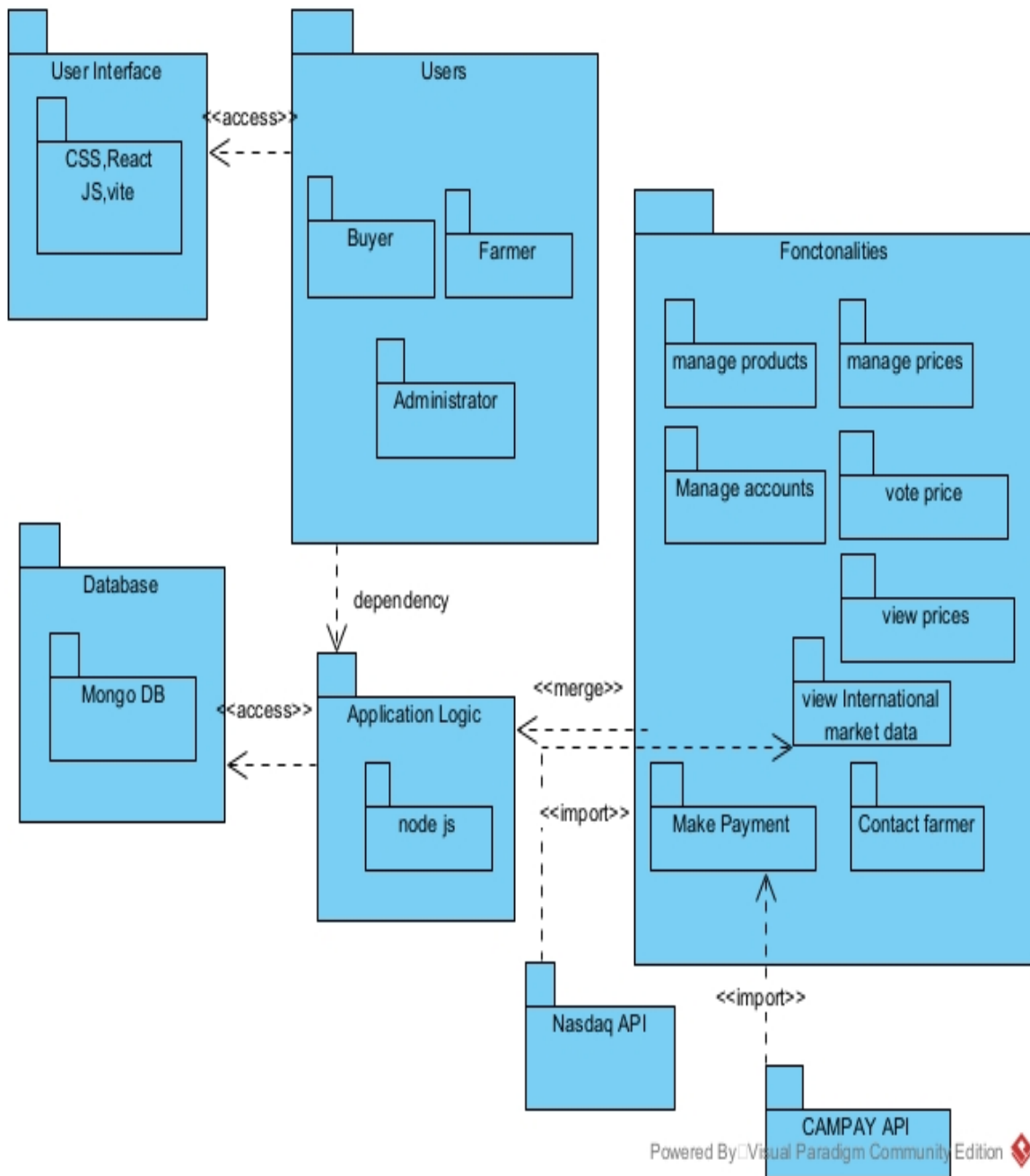


Figure 27: TchopPrice Package Diagram

CONCLUSION

In the conception phase, we set as objective to plane the different aspect of our system by Showing how it will be structure and deployed within existing technical architectures. We began by considering the technical constraints for our system, after which we proceeded to identify the components of our system, how they are grouped together and how they should be deployed on appropriate deployment targets. We finished this phase by looking at interactions between the various aspects and actors of our system. The next phase of our report is the realization phase where we will look at aspects concerning the implementation of our system.

BOOK V: REALIZATION PHASE

Preamble

In this phase we will to straight forward in the implementation of our solution, we will base ourselves on the analysis and conception phases and present the component and deployment diagrams.

Content

INTRODUCTION

- I. DEPLOYMENT DIAGRAM
- II. COMPONENT DIAGRAM
- III. CHOICES OF TECHNOLOGIES

CONCLUSION

INTRODUCTION

Here in the realization phase, we will concentrate on building or implementing our solution, based on the different analysis and conception that we had carried out, which will help to facilitate our work, this phase is as critical as the previous phases. We are going to look at the relationship that exist between the entities of the entity relational diagram. Furthermore, we will see the choices of technologies used for the implementation of our system.

I. DEPLOYMENT DIAGRAM

a. Definition:

Deployment diagram is a structural diagram used to visualize the topology of the physical components of a system, where the software is deployed. They consist of nodes and their relationship. It is related to the component diagram because the components are deployed using the deployment diagram. A deployment diagram consists of nodes. Nodes are nothing but physical hardware used to deploy the application.

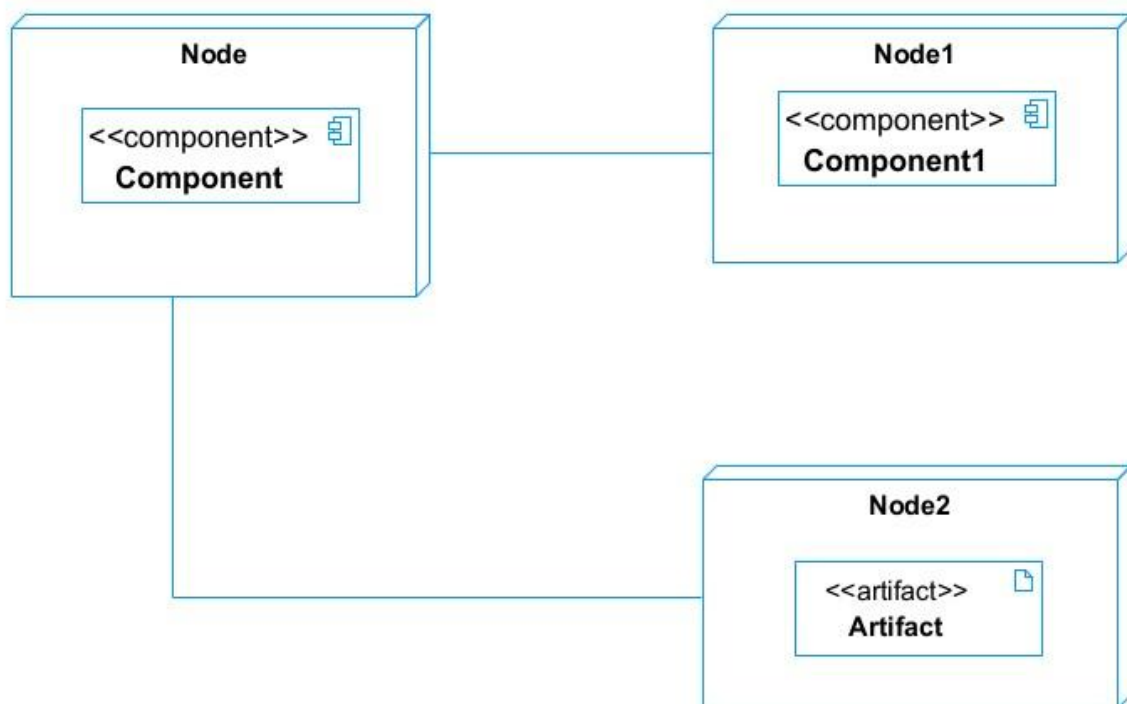
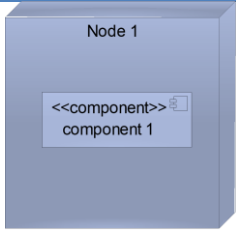
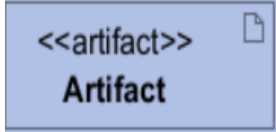
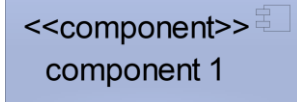


Figure 28: Deployment diagram Formalism

b. Deployment diagram elements

Table 24: Components of a Deployment Diagram

Element	Representation	Description
Node		It is a hardware used to deploy the application
Artifact		An artifact is a major product, which is produced or used during the development of a software. E.g. diagrams, data models, setup scripts
Component		It represents a modular part of a system that encapsulates its content and whose manifestation is replaceable within its environment.
Association		An association helps to connect two nodes together which permits them to communicate together

Deployment Diagram

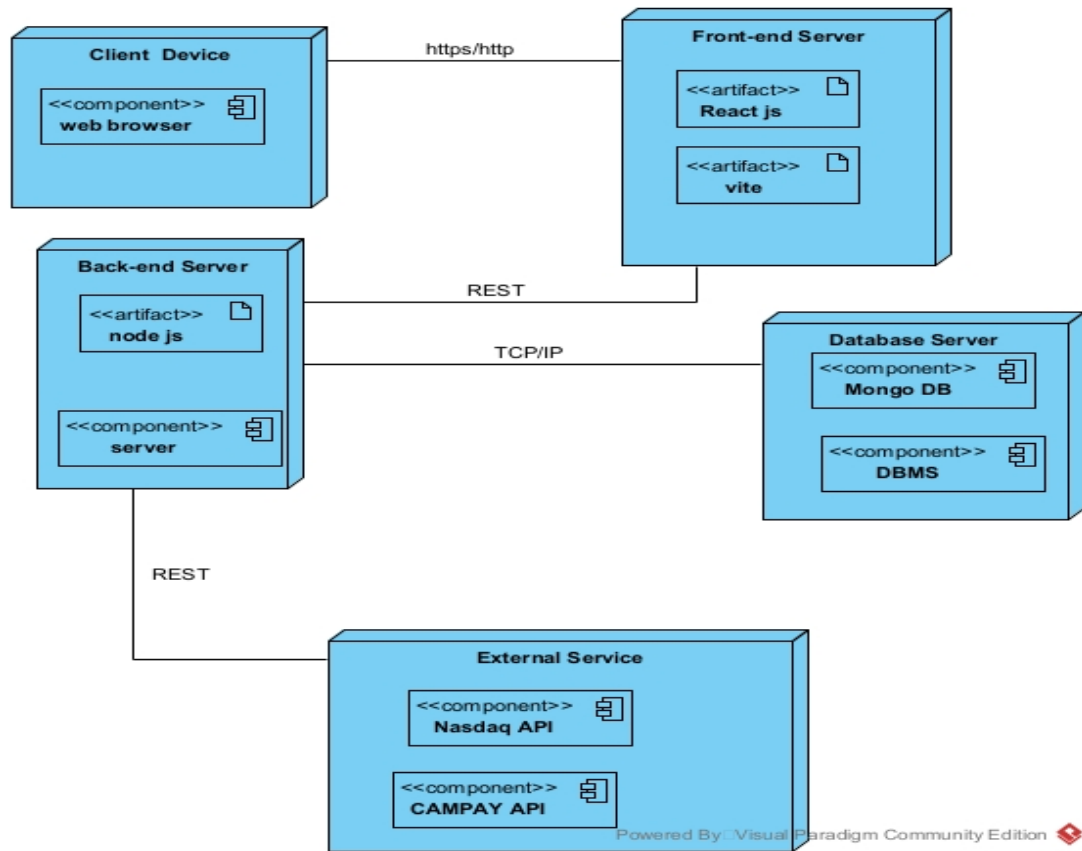


Figure 29: TchopPrice Deployment Diagram

II. Component Diagram:

Definition:

Component diagrams are used to model the physical aspect of a system. Now the question is what are this physical aspect? They are elements such as Executables, libraries, files, document etc. which resides in a node. The component diagram does not describe the functionality of the system but it describes the components used to make those functionalities.

a) Formalism:

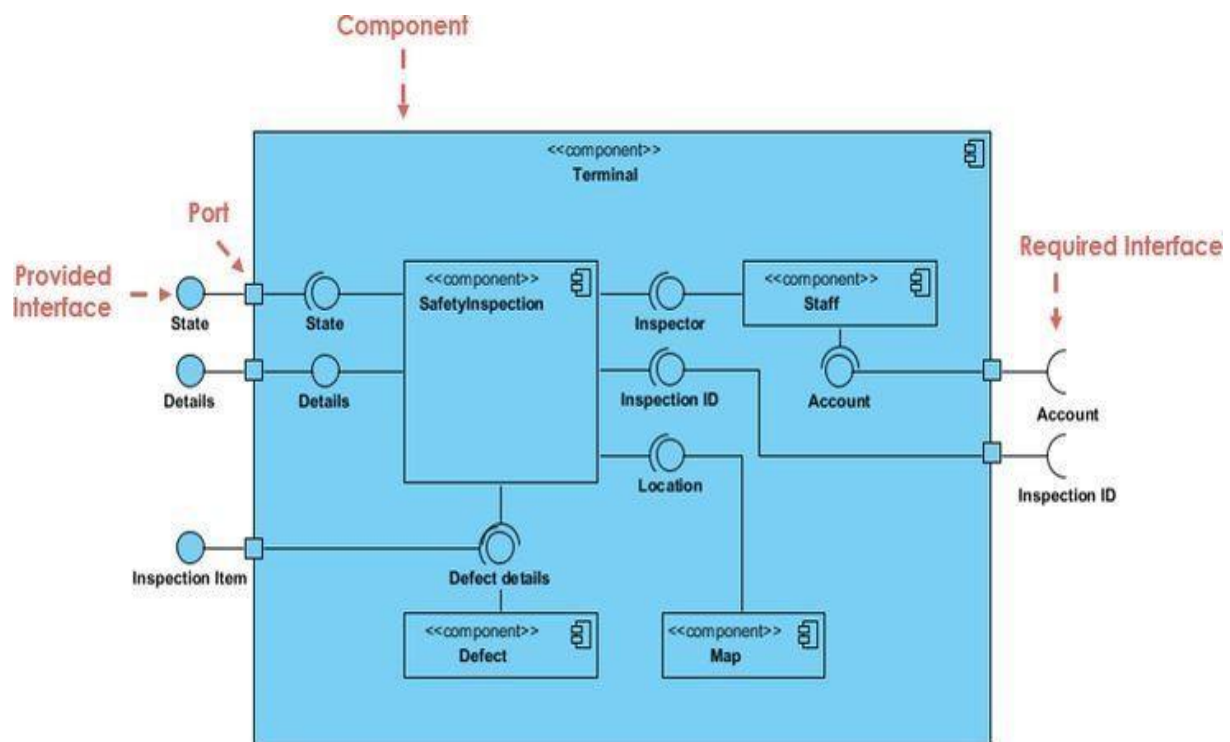
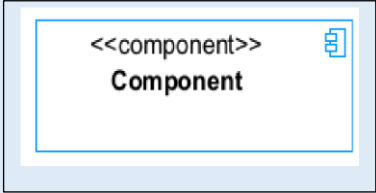
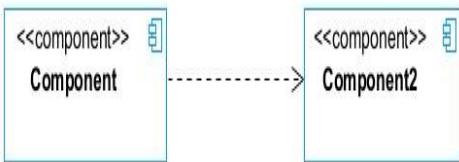
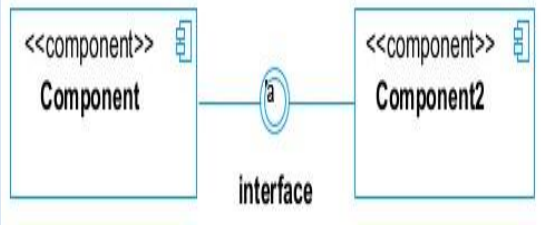
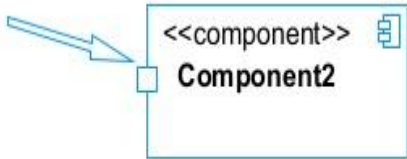


Figure 30: Formalism of a component Diagram

(Source: <https://www.pinterest.com/pin/551128073157994549/>)

Elements of a component diagram

Table 25: Elements of a component diagram

NAME	REPRESENTATION	DESCRIPTION
A component		A component is an abstract logical unit block of a system. It is represented as a rectangle with a smaller rectangle in the upper right corner which serves as its icon for recognition.
Dependency		Dependency is a directed relationship which is used to show that some components are dependent on others for their correct functioning.
interface		An interface is a circle, or a semicircle attached to a stick which looks like a lollipop. It describes groups of operations provided or required by components.
port		A port (represented by a small square at the end of a required or provided interface) is used when the components delegate the interfaces to an internal class.

III. TECHNOLOGICAL CHOICES

1. Software used for the implementation of PIOL

Table 26: Software tools used for the project

Software	Version	Role	Image
WINDOWS 11	10.0.26100	The operating system required to run the software is the one on which we have worked.	
Visual studio code	1.96.2.0	Text editor that permits code writing and debugging line by line	
Postman	9.4	It is a platform that simplifies every step of the API lifecycle and streamlines collaboration, making it easier and faster to create better APIs.	
GitHub	3.0.6	It is a website and cloud service that helps developers store and manage their code.	
Frameworks react JS	18.3.1	React Native is an English framework built on JavaScript for creating Android and iOS applications.	
Visual Paradigm	17.2	Visual Paradigm provides both more formal notation and casual drawing capabilities	

2. Presentation of the languages used

Table 27: Presentation of the languages used for the software

Image	Name	Usage
	Node JS – Framework	Used for the development of the application core/kernel. Backend development
	JavaScript	The programming language used for front-end processing is JavaScript.
	Framework React JS	Front-end development
	Tailwind CSS	Stylesheet for the frontend

CONCLUSION

In our realization phase, we implemented our application. In other to accomplish this phase, we made use of our analysis and conception phase. We also drew the deployment and component diagrams which depict the structure of our system in terms of modules, files, assets, how the different elements interact with each other. We will move to the test of functionalities phase, where we will examine the different modules present in our app and how beneficial they are to its different users

BOOK VI: TEST OF FUNCTIONALITIES

Preamble

In this phase, we will present the various functionalities of our application.

Content

INTRODUCTION

- I. APPLICATION FUNCTIONALITIES
- II. TESTING
- III. TEST SHOW CASES

CONCLUSION

INTRODUCTION

The test of functionalities phase helps us to know more about the solution we are building be it web or mobile. It provides the different functionalities or modules found in our application and how they are beneficial to the users. Hence, we are going to explore the different functionalities present in TchopPrice.

I. Application Functionalities

- **User Authentication & Roles**
 - Users can register and log in as farmers, buyers, or admins.
 - Role-based access: different menus and permissions for each role.
- **Dashboard**
 - Personalized dashboard for each user with relevant stats and quick links.
- **Product Management**
 - Farmers/admins can create, view, update, and delete products.
 - Products can be listed, searched, and filtered.
- **Market Prices**
 - Users can view and possibly report or vote on market prices for products.
- **Voting System**
 - Eligible users can upvote/downvote products or prices (buyers are restricted).
- **Notifications**
 - Users receive notifications for important events (e.g., product approval, price changes).
 - Notifications can be marked as read.
- **Profile Management**
 - Users can view and edit their profile information
 - Admins can manage users, verify/block/unblock users, and view stats.
- **Messaging & Payments**
 - (If implemented) Farmers can access messages and payment features.
- **Help & Settings**
 - Users can access help and settings pages
- **Logout**

II. TESTING

Types of tests:

Some of the common types of tests for web applications include:

- **Functional Testing:** This type of testing focuses on verifying that each function of the web application behaves as expected. It involves testing individual features, user interactions, forms, links, and navigation.
- **Usability Testing:** Usability testing checks how user-friendly the web application is. It involves evaluating the ease of use, intuitiveness, and overall user experience of the application.
- **Compatibility Testing:** Compatibility testing ensures that the web application works correctly across different web browsers, operating systems, and devices. It helps identify any compatibility issues or inconsistencies.
- **Performance Testing:** Performance testing evaluates how the web application performs under various conditions. It involves measuring response times, load handling capabilities, scalability, and the overall system performance.
- **Security Testing:** Security testing aims to identify vulnerabilities and weaknesses in the web application's security. It includes checking for potential threats such as SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and other security vulnerabilities.
- **Integration Testing:** Integration testing verifies the interactions between different components, modules, or services within the web application. It ensures that they work together seamlessly, and that data is transferred correctly between them.
- **Regression Testing:** Regression testing is performed to ensure that new changes or updates to the web application do not introduce any new issues or break existing functionality. It involves retesting previously tested features and functionalities.
- **Accessibility Testing:** Accessibility testing checks whether the web application is accessible to users with disabilities. It ensures compliance with accessibility guidelines and standards, making the application usable for all users.

Test Showcase

The following are the tests carried out using Postman on our APIs' endpoints

- Testing login endpoints

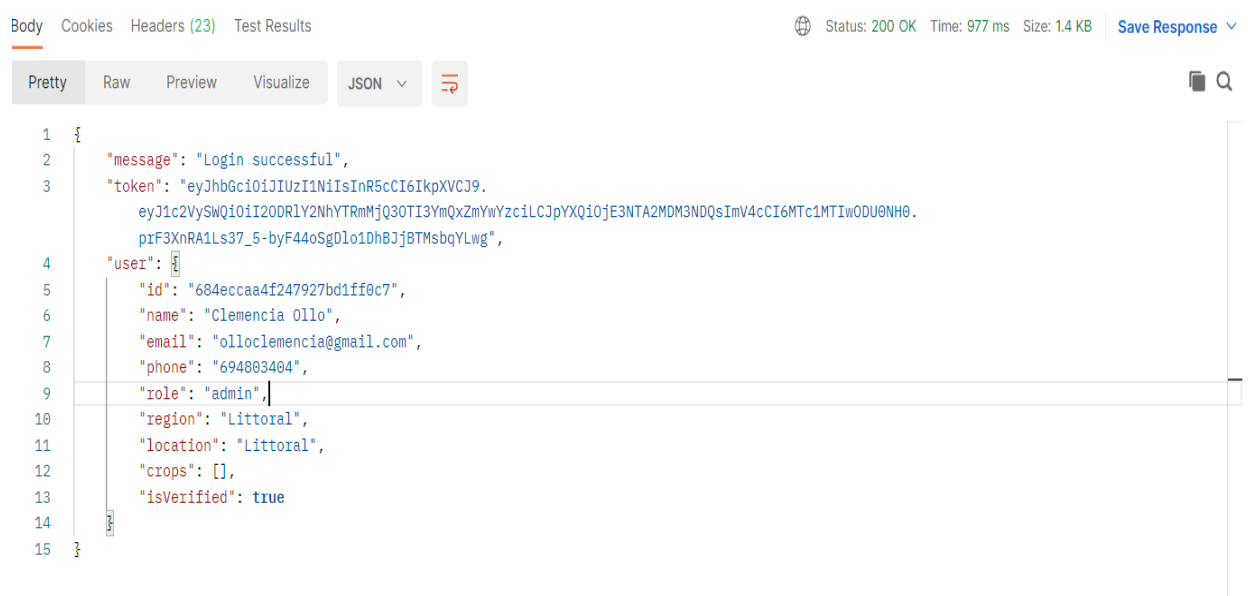


Figure 31: login test

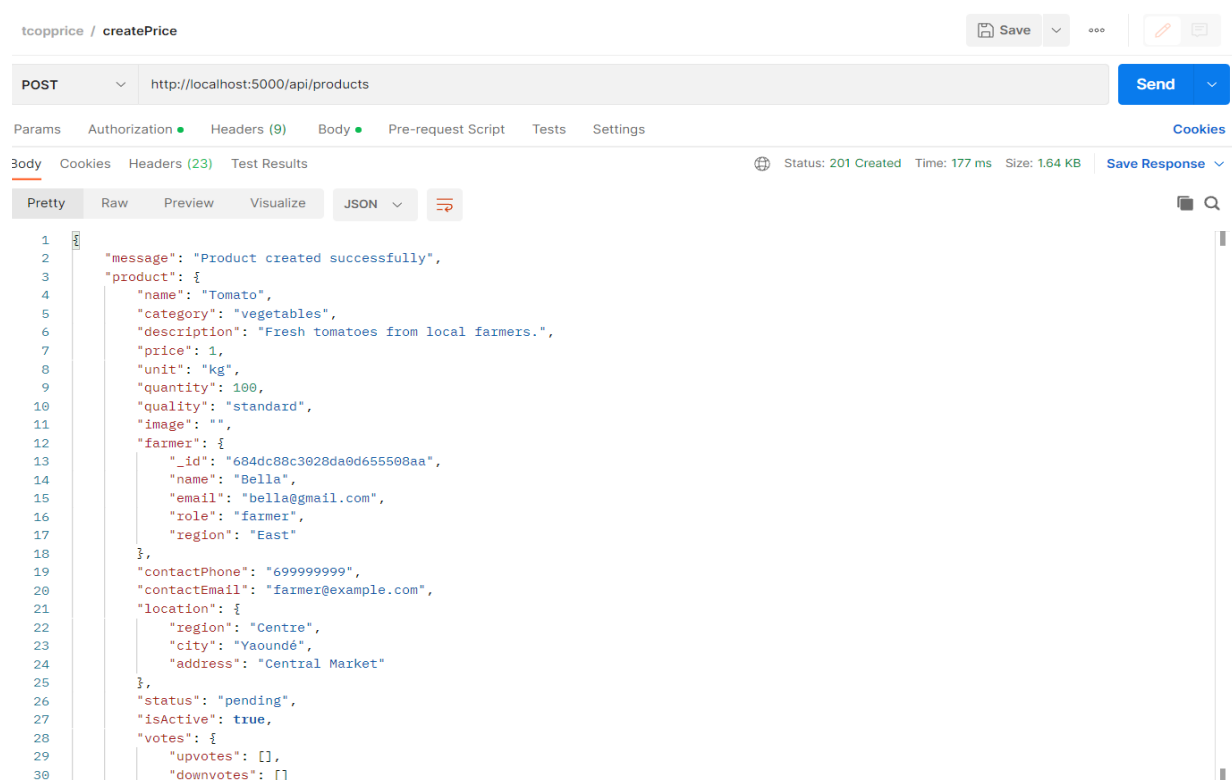


Figure 32: create property test

BOOK VII: USER GUIDE

Preamble

The purpose of the user guide is to provide users of our platform with step-by-step instructions on how to install and use the system.

Content

INTRODUCTION

I. INSTALLATION OF THE APPLICATION

II. SHOW CASE

CONCLUSION

INTRODUCTION

This is the final phase of our report. In this phase we will walk through the requirements for our system, the necessary installation process, accessing our system and its features, all this in a step-by-step manner to facilitate the setting up of the platform for the first-time users. The steps of different processes will be accompanied by images. After we will showcase our application by viewing the different screens

I. INSTALLATION OF THE APPLICATION

A. How to install MongoDB

To deploy a software, we need to install its database server, in our case MongoDB will be installed in our system which is Windows.



Figure 33: MongoDB Logo

STEP 1 : Downloading the MongoDB MSI installer package

We go to <https://www.mongodb.com/try/download/community> in order to download MongoDB and note that we are downloading the community version since it is free and open source.

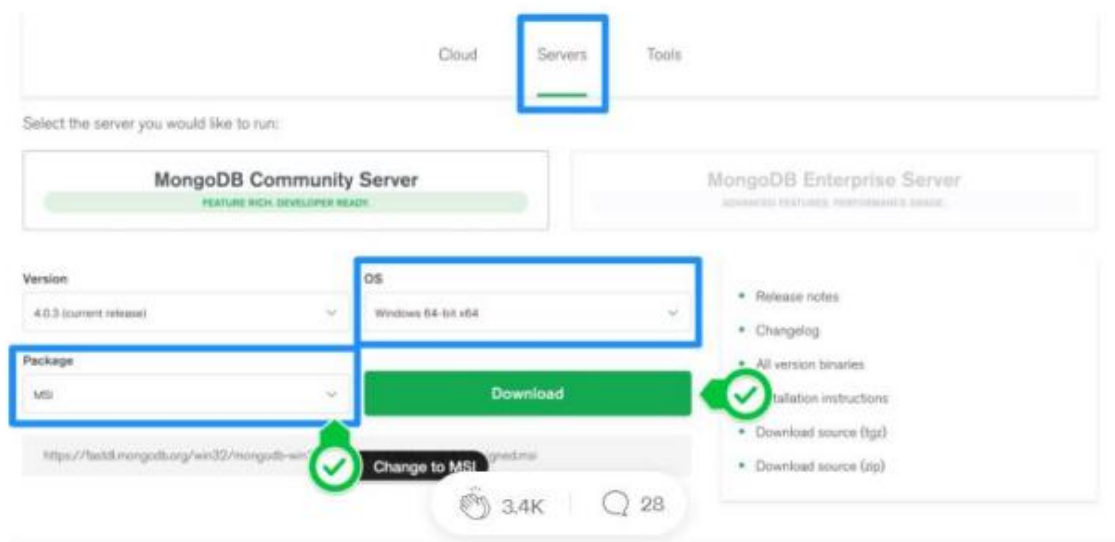


Figure 34 Downloading MongoDB community version

STEP 2: Install MongoDB with the installation Wizard

- A. Make sure you logged in as a user with admin privileges. Then navigate to your downloads folder and double click on the. MSI package you just downloaded. This will launch the installation wizard.

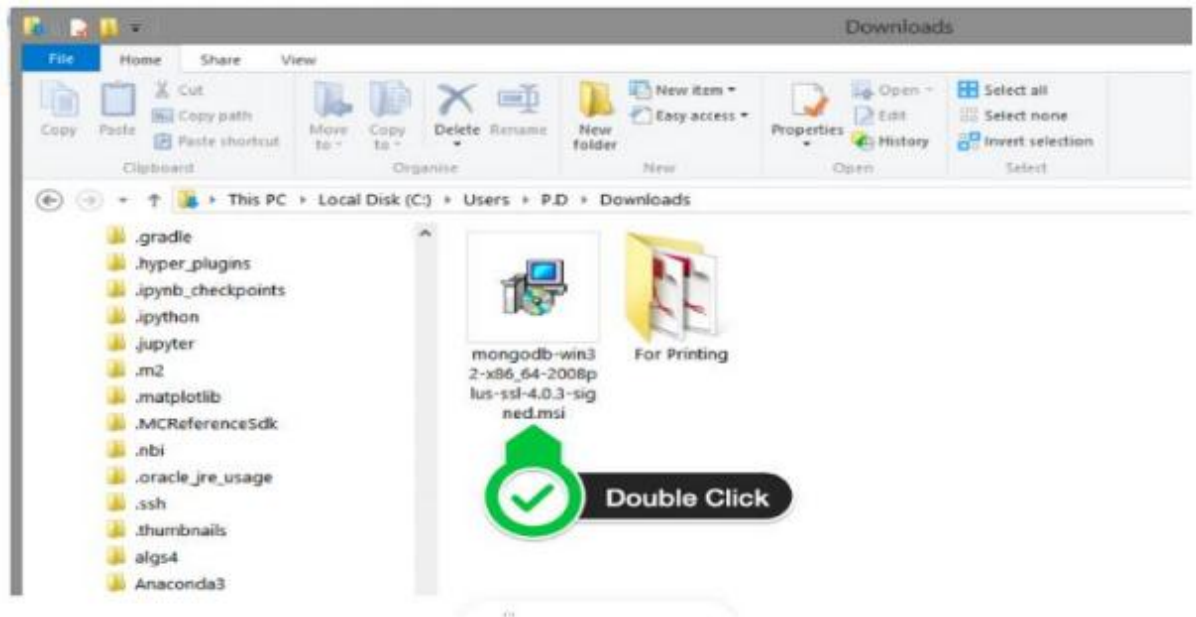


Figure 35 Launching the installation wizard

- B. Click on the next button



Figure 36 Clicking on next

C. Accept the license agreement then click Next

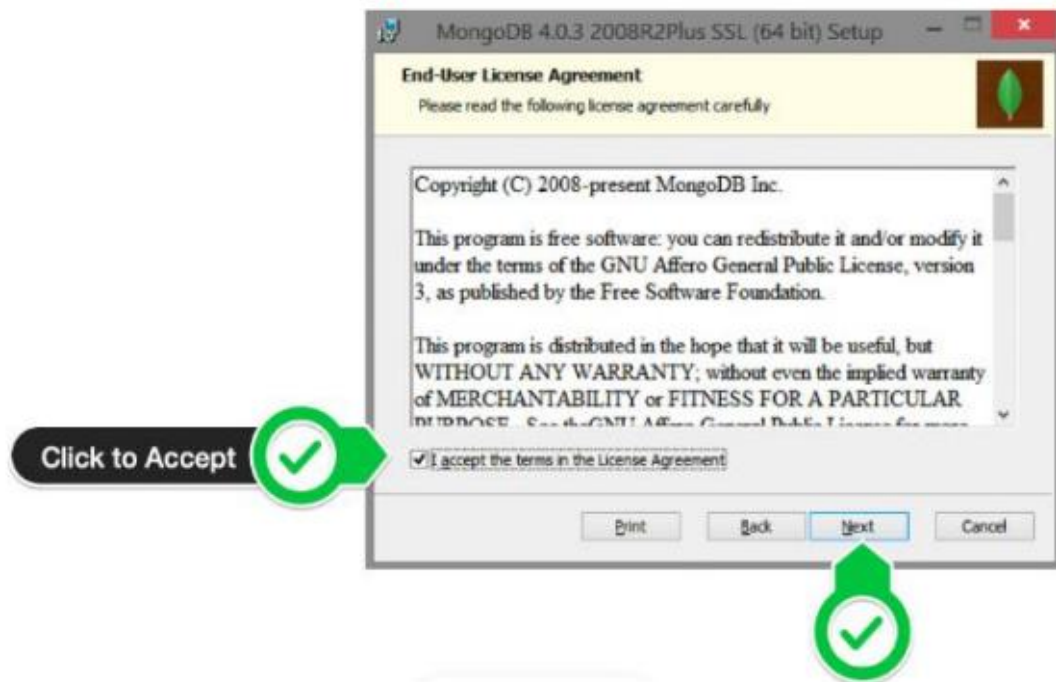


Figure 37: Agreement license

D. Select the Complete setup

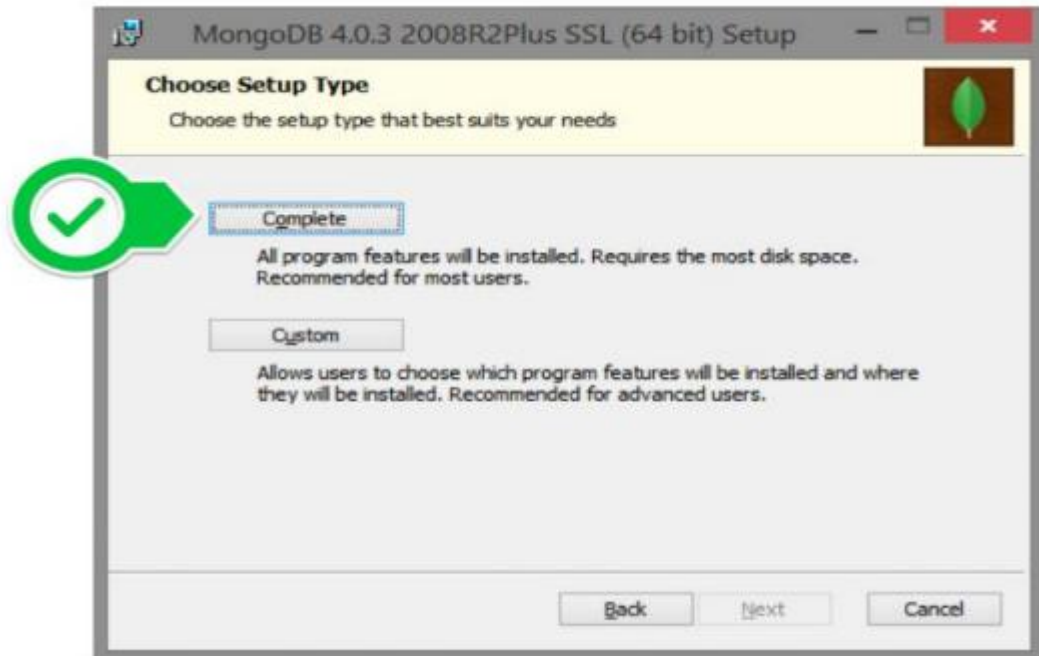


Figure 38: Select complete setup

- E. Select “Run service as Network Service user” and make a note of the data directory, we’ll need this later.

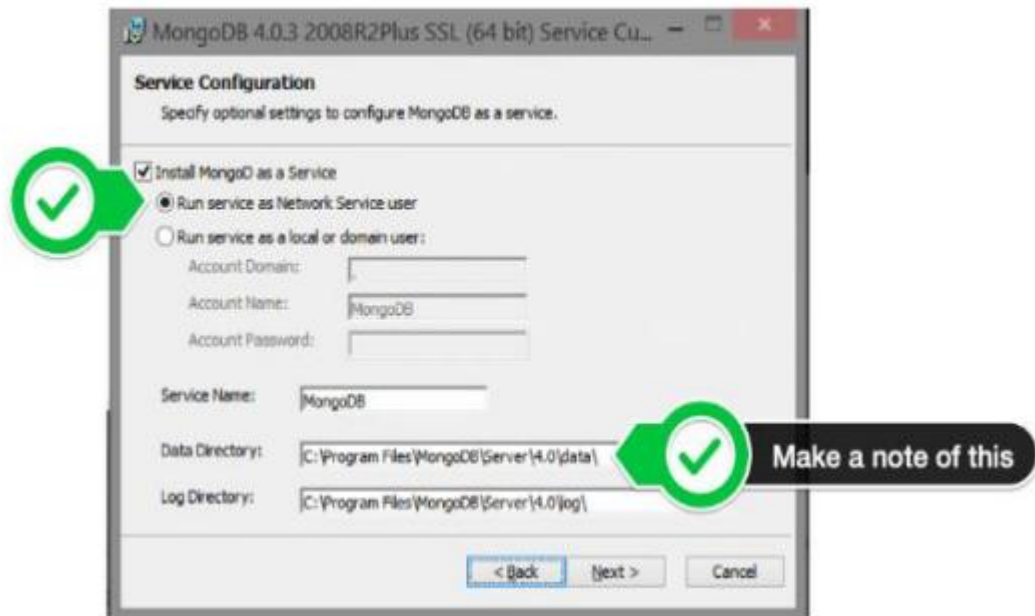


Figure 39: Run Service network user

- F. Select mongo DB compass since you don’t have that already

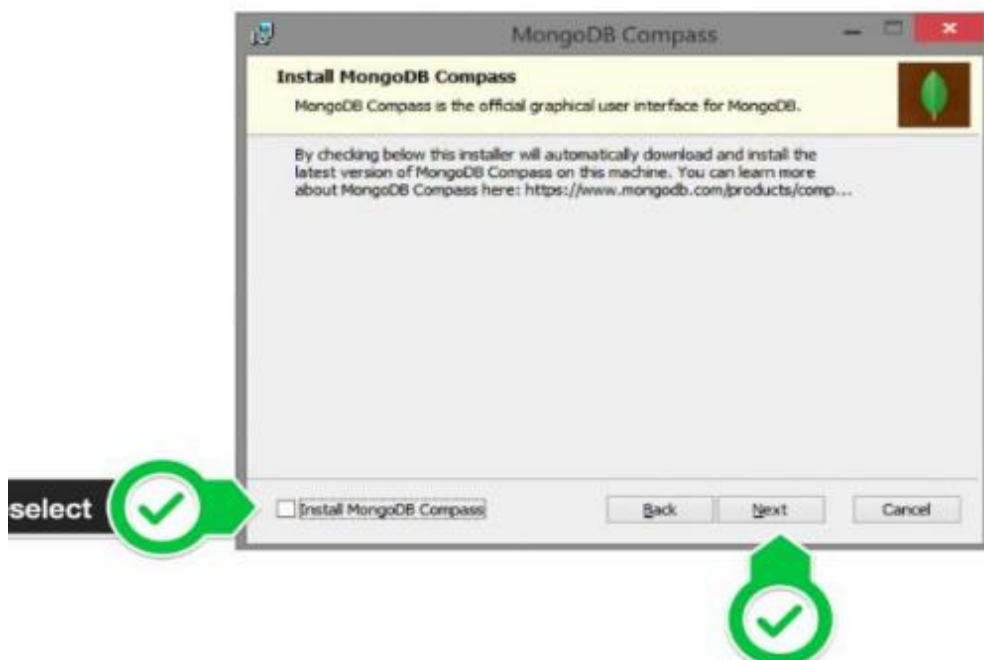


Figure 40: Selecting MongoDB compass and clicking next

G. Click Install to begin installation

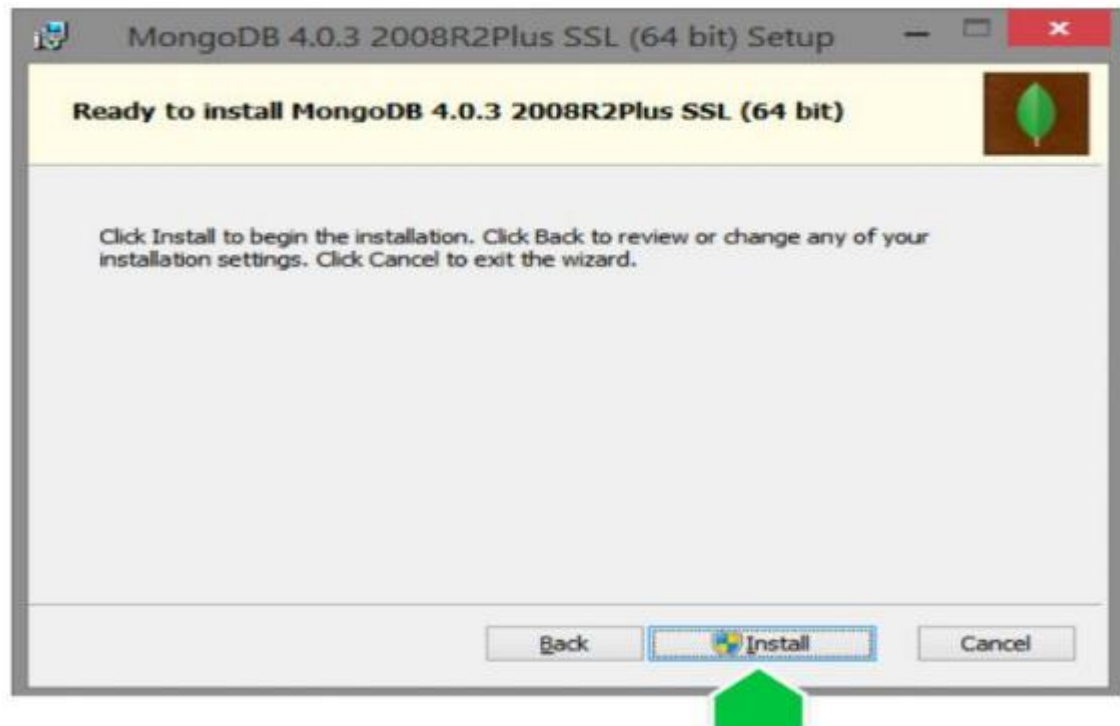


Figure 41: Click on install to start installation

H. Click on finish to complete installation



Figure 42: Click on finish to complete installation

WEB APPLICATION PRESENTATION

SHOWCASES

1. Home Page

The Home Page welcomes users with a clean interface, showcasing different sections. Users can navigate through the site using the responsive navigation bar at the top.

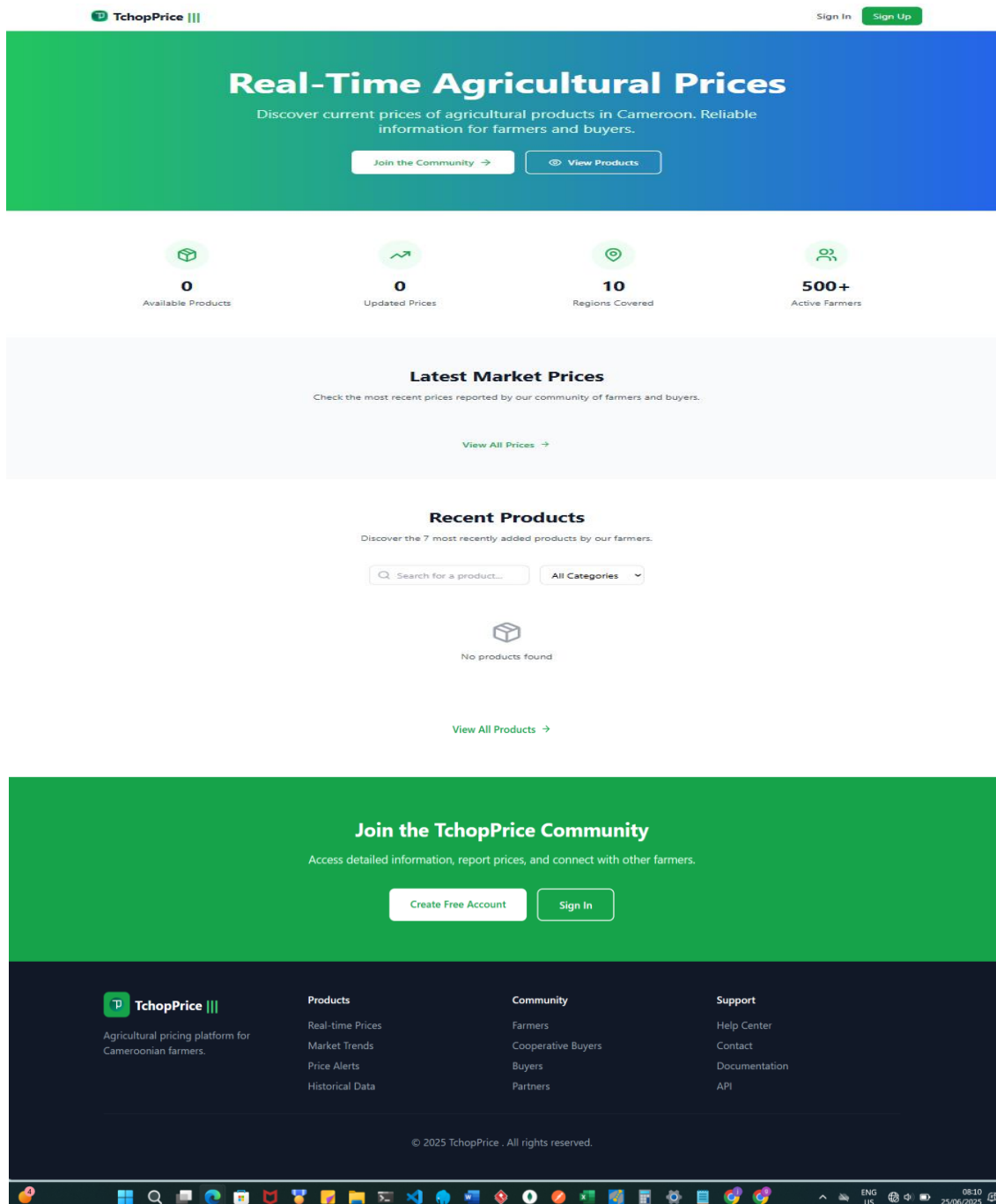
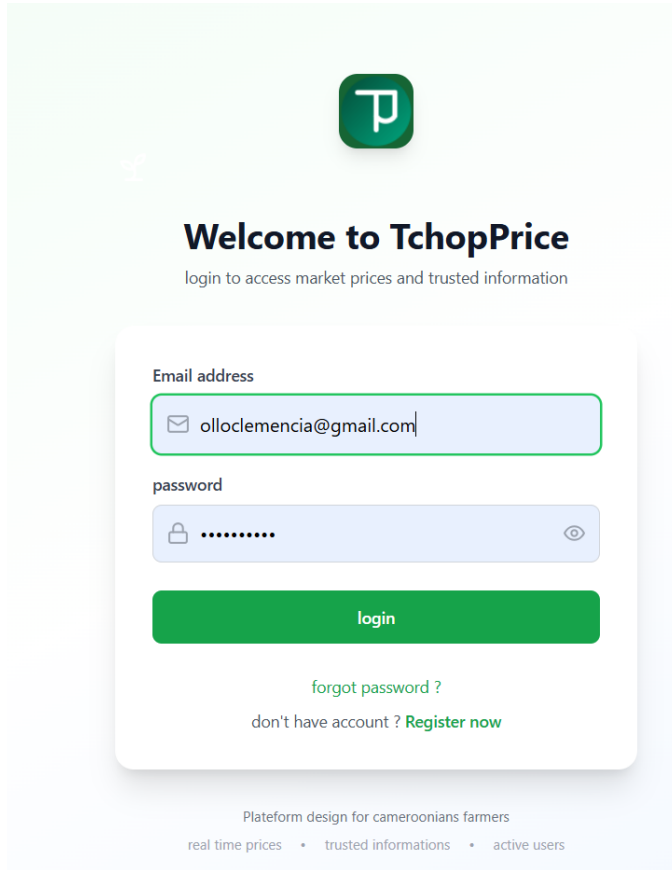


Figure 43: Landing Page

2. Login Page

This interface is our login page. If we already have an account, we can simply enter our credentials (email and password) and click on “Login” to submit.



The screenshot shows a login page for 'TchopPrice'. At the top, there is a green square logo with a white 'T' and a checkmark. Below the logo, the text 'Welcome to TchopPrice' is displayed in bold, followed by the subtitle 'login to access market prices and trusted information'. The login form consists of two input fields: 'Email address' with the value 'olloclemencia@gmail.com' and 'password' with masked characters. A green 'login' button is positioned below the password field. Below the button, there are two links: 'forgot password ?' and 'don't have account ? Register now'. At the bottom of the page, a footer line reads 'Plateform design for cameroonians farmers' followed by three bullet points: 'real time prices', 'trusted informations', and 'active users'.

Figure 44: login page

3. Dashboard Page

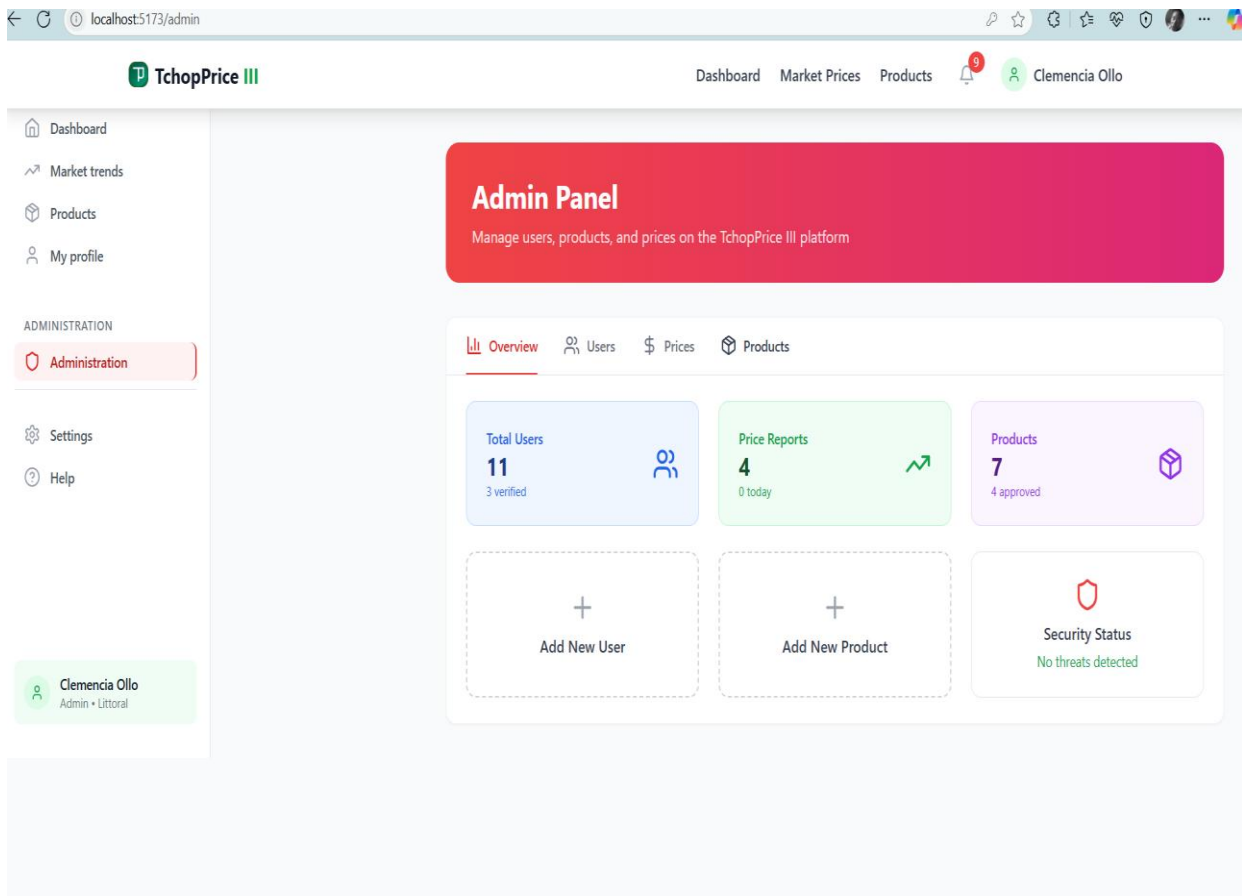


Figure 45: Admin dashboard

CONCLUSION

Having put in place the platform, it was not sufficient for we had to produce a manual that will help its various users. That is why we presented the different tools to be installed and how they are to be installed to run this application without any problem and how the users will use this platform once the environment is set up.

PERSPECTIVES

Considering the fact this project can be improved to a higher extend, and considering the lack of some skills and understanding, as perspectives, I can bring some ameliorations such as;

- Improve a chat system to ease the communication between users (Farmers and buyers)
- Integration of AI for fast decision making
- Implementing cloudinary for image uploads.
- Implementing the charts and graphs to easily track market price insights.
- Improvements on the user interface (dashboards, Agent page, Property page, etc...)

GENERAL CONCLUSION

Having come to an end of our project, we learned how to manage stress and time with multiple school projects and exams. For our project we worked on the theme “**DESIGN AND IMPLIMENTATION OF A FARMER CENTRIC WEB APPLICATION FOR AGRICULTURAL PRICE MONITORING**”, our greatest desire is to empower Cameroonian farmers, that provides real-time, reliable, and region-specific **agricultural product price information** to support better decision-making and reduce economic exploitation. We began by identifying the current system in place for the consultation process, project constraints and requirements in the specification book, then we went forth to analyse our system (TchopPrice) using UML-2TUP methodology. We made use of visual paradigm, a modelling software used to draw our various diagrams. To realize our project, we used react-native for mobile front-end, React js for the front-end, NodeJS for the backend and MongoDB as our database. Our system, TchopPrice will enable users to use **real-time dashboards** displaying prices for major agricultural products across regional markets., receive notifications or reminders for new prices. We won't end here, updates and important improvements are going to be made in the nearest future in terms of functionality, security to make it more reliable.

ANNEXES

BIBLIOGRAPHY

1. **Booch, G., Rumbaugh, J., & Jacobson, I.** (2005). *The Unified Modelling Language User Guide*. 2nd Edition. Addison-Wesley.
2. **Fowler, M.** (2003). *UML Distilled: A Brief Guide to the Standard Object Modelling Language*. 3rd Edition. Addison-Wesley.
3. **Ambler, S. W.** (2009). *Agile Modelling: Effective Practices for Extreme Programming and the Unified Process*. Wiley.
4. **Pressman, R. S.** (2014). *Software Engineering: A Practitioner's Approach*. 8th Edition. McGraw-Hill Education.
5. **Larman, C.** (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd Edition. Pearson.

WEBOGRAPHY

- + Cameroon Ministry of Agriculture and Rural Development (MINADER) – Official price bulletins and agricultural reports
<https://www.minader.cm/index.php/category/prix-des-marches/>
- + Mercuriale de Prix Cameroun 2024 (Scribd PDF Reference),
<https://www.scribd.com/document/561202966/mercuriale-2024>
- + MongoDB – Official documentation for NoSQL database usage,
<https://www.mongodb.com/docs/>
- React.js – Official documentation for front-end development, <https://reactjs.org>
- + Vite – Fast frontend build tool and development server <https://vitejs.dev/>
- Node.js – Server-side runtime environment <https://nodejs.org>
- Visual Paradigm – UML modelling platform, <https://www.visual-paradigm.com/>
- + Postman – API testing platform, <https://www.postman.com/>
- + National Institute of Statistics (Cameroon) – Inflation and food price trends
<https://ins-cameroun.cm/>
- + Cameroon Marketplace Blog – Agricultural sector and policy challenges
<https://cameroon-marketplace.com/en/blogs/cameroon-marketplace>

TABLE OF CONTENTS

DEDICATION	1
ACKNOWLEDGEMENT.....	2
ABREVIATIONS	4
LIST OF FIGURES.....	5
LIST OF TABLES	7
ABSTRACT	8
Résumé	9
GENERAL INTRODUCTION	10
BOOK I : EXISTING SYSTEM.....	27
I.PRESENTATION OF THE PROJECT THEME	30
II.DESCRPTION OF THE EXISTING SYSTEM	31
III.LIMITATIONS OF THE EXISTING SYSTEM	33
IV.PROBLEMATIC.....	34
V.PROPOSED SOLUTION.....	35
VI.CONCLUSION	36
BOOK II : SPECIFICATION BOOK.....	37
INTRODUCTION.....	39
I.CONTEXT AND JUSTIFICATION OF STUDIES.....	40
II.OBJECTIVES OF THE PROJECT	42
I.EXPRESSION OF NEEDS	43
II.ESTIMATED COST OF THE PROJECT.....	45
A.Hardware Resources	45
B.SOFTWARE RESOURCES.....	46
C.HUMAN RESOURCES	47
D.GLOBAL ESTIMATION	48
III.PROJECT PLANNING.....	49
a)Chronogram of activities.....	49
b)GANTT CHART.....	50
IV.PROJECT CONSTRAINTS	51
V.LIST OF PARTICIPANTS AND DELIVERABLES	52
a)List of participants.....	52
b)Deliverables	52
CONCLUSION	53
BOOK III : ANALYSIS BOOK	54
Preamble.....	55

Content overview	55
INTRODUCTION.....	56
a.Structural Diagrams	58
b.Behavioural Diagrams	58
a.The left branch (Functional branch)	62
b.The right branch (Technical branch)	62
c.The middle branch (Realisation branch).....	63
CONCLUSION	87
BOOK IV : CONCEPTION BOOK.....	88
INTRODUCTION.....	90
CLASS DIAGRAM	97
State Machine Diagram	99
State Machine Diagram	101
CONCLUSION	106
BOOK V: REALIZATION PHASE	107
INTRODUCTION.....	109
CONCLUSION	117
BOOK VI: TEST OF FUNCTIONALITIES	118
INTRODUCTION.....	120
BOOK VII: USER GUIDE	124
INTRODUCTION.....	126
I.INSTALLATION OF THE APPLICATION.....	127
CONCLUSION	135
PERSPECTIVES.....	136
GENERAL CONCLUSION.....	137
ANNEXES	xiii
BIBLIOGRAPHY	xiv
WEBOGRAPHY	xv